Learning-Augmented Smooth Integer Programs with PAC-Learnable Oracles

Hao-Yuan He Jiang-Tian Xue Ming Li

National Key Laboratory for Novel Software Technology
School of Artificial Intelligence, Nanjing University
{hehy, xuejt, lim}@lamda.nju.edu.cn

Abstract

This paper develops learning-augmented algorithms for *smooth integer programs*, a broad class of Boolean polynomial optimization problems that captures canonical combinatorial optimization tasks such as MAX-CUT and MAX-k-SAT. We study an *end-to-end* setting in which a learned oracle predicts a full Boolean assignment for each instance. We introduce Leap, an approximation pipeline that uses the prediction as a reference point for a linear surrogate of the objective, optimizes the surrogate by linear programming, and rounds the result to an integral solution. For objectives of fixed degree, Leap queries the oracle once and runs in polynomial time. We establish two main guarantees. First, Leap degrades gracefully with prediction errors. We bound its loss by an *error footprint*, the part of the decomposed objective that the errors can affect. For MAX-k-SAT with ε prediction errors, this yields an approximation ratio of $1 - O(\varepsilon/n^{\gamma})$ on near-dense instances whose optimum is $\Omega(n^{k-1+\gamma})$. When each literal occurs in at most Δ clauses, the ratio is $1 - O(\Delta\varepsilon/n)$. Second, the oracle is learnable from sampled instances. We bound the pseudo-dimension of the cost class induced by Leap, which gives a PAC-style guarantee for empirical risk minimization. Together, the two guarantees give an end-to-end bound that separates the loss of the best oracle in the class from the statistical error of learning. Experiments on MAX-CUT and MAX-SAT show that Leap substantially reduces the loss of imperfect predictions and that empirical selection identifies useful oracle candidates.

Keywords: smooth integer programs, learning-augmented algorithms, combinatorial optimization, approximation algorithms, PAC learnability

1 Introduction

Combinatorial optimization is a central task in computer science, operations research, and many scientific applications. Canonical problems such as MAX-CUT and MAX-k-SAT ask for the maximization of an objective over a discrete feasible set, and are typically NP-hard; many are also APX-hard (Papadimitriou and Yannakakis, 1991), ruling out a polynomial-time approximation scheme unless P = NP (Arora et al., 1998). In many application settings, however, optimization instances are not encountered in isolation: similar instances are solved repeatedly, creating an opportunity to learn predictions from past data and use them to guide future optimization. This is the premise of *learning-augmented algorithms*, which use data-driven predictions while retaining provable performance guarantees (Lykouris and Vassilvitskii, 2021; Mitzenmacher and Vassilvitskii, 2022).

In this work, we focus on *smooth integer programs*: the maximization of an n-variate degree-d smooth polynomial $p(\mathbf{x})$ over Boolean assignments,

$$\begin{aligned} \max_{\mathbf{x}} \quad & p(\mathbf{x}) \\ \text{s.t.} \quad & \mathbf{x} \in \{0,1\}^n. \end{aligned} \tag{d-IP}$$

Informally, smoothness bounds the coefficient of each degree-ℓ monomial by $O(n^{d-\ell})$.[1] This model was introduced by Arora et al. (1999) and is broad enough to capture MAX-SNP problems (Papadimitriou and Yannakakis, 1991), including canonical graph partitioning tasks such as MAX-CUT and Boolean constraint satisfaction problems such as MAX-k-SAT.

Arora et al. (1999) developed PTASs for *dense* smooth integer programs, where $\mathrm{OPT} = \Omega(n^d)$, using exhaustive sampling, linear programming, and rounding. A key insight in their analysis is that local coefficients estimated from a partial assignment can turn the nonlinear objective into a linear one. Because an optimal assignment is unknown, their algorithm samples a small set of variables and enumerates all Boolean assignments on the sample, which takes $n^{O(1/\eta^2)}$ time for a target precision η.

More recently, Bampis et al. (2024) incorporated partial predictions into this sampling-and-LP paradigm through a *parsimonious* oracle. Their oracle predicts labels only on a sampled set of variables, which replaces the enumeration step, and the guarantee degrades with the number of wrong sampled predictions. Like that of Arora et al. (1999), their guarantee applies to *dense* objectives. This result treats the oracle as an external source of advice, leaving open how the oracle itself is obtained.

Our goal is to build an end-to-end learning-augmented framework for smooth integer programs, in which the oracle itself is learned from data. Learned predictors for graph and formula instances output a label for every variable (Selsam et al., 2019; Pan et al., 2025), so we let the oracle predict a full Boolean assignment for the instance. A full prediction determines all local coefficients at once, which removes the sampling step of both methods above and the enumeration step of Arora et al. (1999). This end-to-end viewpoint leads to two natural questions that organize the paper:

Q1. *Given a predicted assignment that may contain errors, can we turn it into an approximation algorithm whose guarantee degrades gracefully with the prediction error?*

Q2. *Can one learn such a predictor from sampled instances, so that the resulting algorithm has a PAC-style performance guarantee?*

1.1 Framework and Main Results

We answer these questions through a learning-augmented approximation framework, using MAX-k-SAT as a concrete running example. Consider a MAX-k-SAT formula with clause set $\mathcal{C}$, where each clause contains exactly k literals on distinct variables. For a clause C, let C^+ and C^- denote its positive and negative literal indices. The number of satisfied clauses is the Boolean polynomial

$$p(\mathbf{x}) = \sum_{C \in \mathcal{C}} \left(1 - \prod_{i \in C^-} x_i \prod_{i \in C^+} (1 - x_i) \right).$$

The product term is one exactly when clause C is falsified. The objective is a degree-k smooth integer program: each degree-ℓ coefficient is $O(n^{k-\ell})$.

[1] In our results, we treat the degree d as a fixed constant, and $O(\cdot)$ and $\Omega(\cdot)$ hide factors that depend only on d.

For Q1, we develop LEAP, short for LEarning-augmented Approximation via linear Programming. Given an instance π, an oracle f predicts a full Boolean assignment $f(\pi) = \widehat{\mathbf{x}} \in \{0,1\}^n$. LEAP uses this prediction as a reference point and proceeds in three steps.

(1) **Linearize.** Decompose the objective recursively into variables multiplied by lower-degree local coefficients, and evaluate these coefficients at $\widehat{\mathbf{x}}$ to obtain a linear surrogate. For example, a clause $x_1 \vee x_2$ contributes $x_1 + x_2 - x_1x_2 = x_1(1-x_2) + x_2$ to p. Evaluating the local coefficient $1 - x_2$ at $\widehat{\mathbf{x}}$ gives the linear term $x_1(1-\widehat{x}_2)$.

(2) **Relax and round.** For an error budget ε, optimize the surrogate by a linear program that tolerates up to ε wrong coordinates in $\widehat{\mathbf{x}}$, and round the fractional solution to an integral assignment. Since p is multilinear, rounding the coordinates one at a time, each to the better of 0 and 1, does not decrease p; we call this *deterministic rounding*.

(3) **Select.** Since the prediction error is unknown, repeat step (2) over error budgets[2] and return the best rounded assignment.

Given an oracle prediction $\widehat{\mathbf{x}}$, let $\mathbf{z}$ be the assignment returned by LEAP. Its loss relative to OPT is bounded by a footprint budget that depends on the prediction error in $\widehat{\mathbf{x}}$.

Theorem 1.1. *Consider a MAX-k-SAT instance with objective p, and let $\mathbf{x}^*$ be an optimal assignment. Suppose the oracle prediction $\widehat{\mathbf{x}}$ differs from $\mathbf{x}^*$ in at most ε coordinates.* LEAP *returns an assignment $\mathbf{z}$ satisfying*

$$p(\mathbf{z}) \geq OPT - O(\Phi_\varepsilon(\widehat{\mathbf{x}})).$$

Here $\Phi_\varepsilon(\widehat{\mathbf{x}})$ is the footprint budget: it measures how much of the decomposed objective can be affected by up to ε prediction errors around $\widehat{\mathbf{x}}$.

With a perfect prediction, $\Phi_0(\widehat{\mathbf{x}}) = 0$, and LEAP returns an optimal assignment.

As the prediction error grows, we bound the footprint budget in two ways. A global bound, which uses only the smoothness scale of the objective, gives a *near-dense* statement. LEAP returns an assignment satisfying

$$p(\mathbf{z}) \geq \mathrm{OPT} - O(n^{k-1}\varepsilon).$$

If $\mathrm{OPT} = \Omega(n^{k-1+\gamma})$ for a constant $\gamma \in [0,1]$, the approximation ratio is $1 - O(\varepsilon/n^\gamma)$.

A local bound, which uses instance structure, gives a *bounded-occurrence* statement by charging each prediction error to only $O(\Delta)$ affected clauses. If each literal occurs in at most Δ clauses, then LEAP returns an assignment satisfying

$$p(\mathbf{z}) \geq \mathrm{OPT} - O(\Delta\varepsilon).$$

Since $\mathrm{OPT} = \Omega(n)$ once variables that appear in no clause are removed, the approximation ratio is $1 - O(\Delta\varepsilon/n)$.

Both statements extend beyond the dense regime $\mathrm{OPT} = \Omega(n^k)$ studied by Arora et al. (1999) and Bampis et al. (2024) to instances with $\mathrm{OPT} = o(n^k)$. For the near-dense regime, Fotakis et al. (2016) give a sampling-based approximation scheme that runs in subexponential time; given a prediction, LEAP runs in polynomial time, with a guarantee that depends on the prediction error.

We next turn from analyzing a given oracle prediction to selecting an oracle from sampled instances. For Q2, we establish a PAC-style guarantee for oracle selection using the algorithm-selection framework of Gupta and Roughgarden (2017). Instances are drawn from an unknown

[2]The budgets can be $\{0,1,\ldots,n\}$ or the doubling grid $\{0,1,2,4,\ldots\} \cup \{n\}$ of $O(\log n)$ budgets; the latter changes the guarantees only by a constant factor (proposition 3.1).

distribution, and each predictor f induces a LEAP algorithm. We choose a predictor by empirical risk minimization (ERM) over the cost of the solutions returned by these induced algorithms, and bound the pseudo-dimension of the resulting cost class. Let $\mathcal{P}(f)$ denote the expected objective value of the solution that LEAP returns with predictor f, and let H upper-bound objective values. For MAX-k-SAT, $\mathcal{P}(f)$ is the expected number of satisfied clauses, and H may be taken as an upper bound on the number of clauses.

The following end-to-end bound combines an ERM guarantee for the induced cost class with the footprint theorem.

Theorem 1.2. *Fix $\alpha > 0$ and $\delta \in (0, 1]$. Let $\mathcal{F}$ be a predictor class, and let $d_{\mathcal{F}}$ be the sum of the VC dimensions of the n classes formed by its coordinate functions. Suppose ERM selects $\hat{f} \in \mathcal{F}$ from m sampled instances, where*

$$m = \Omega\left(\left(\frac{H}{\alpha}\right)^2 \left(d_{\mathcal{F}} \log(e d_{\mathcal{F}}) + \log\frac{1}{\delta}\right)\right).$$

Then, with probability at least $1 - \delta$,

$$\mathcal{P}(\hat{f}) \geq \mathcal{P}^* - O(\Lambda^{\Phi}_{\mathcal{F}}) - 2\alpha.$$

Here $\mathcal{P}^*$ denotes the expected optimum, and $\Lambda^{\Phi}_{\mathcal{F}}$ is the footprint loss of the best predictor in $\mathcal{F}$, defined formally in section 4. The bound separates this footprint loss from the statistical error 2α of ERM. For MAX-k-SAT, the two footprint bounds above give $\Lambda^{\Phi}_{\mathcal{F}} = O(n^{k-1}\varepsilon_{\mathcal{F}})$ under the global smoothness bound and $\Lambda^{\Phi}_{\mathcal{F}} = O(\Delta\varepsilon_{\mathcal{F}})$ under bounded occurrence, where $\varepsilon_{\mathcal{F}}$ is the best expected prediction error achieved by a predictor in $\mathcal{F}$.

1.2 Our Contributions

Our main contributions are as follows.

1. *A learning-augmented approximation pipeline for smooth integer programs.* We introduce LEAP, which turns a predicted assignment into a linear surrogate of a smooth polynomial objective, optimizes it by linear programming, and rounds the result, using one oracle query and polynomial time (section 3).

2. *Error-footprint guarantees.* We develop an error-footprint analysis that bounds the loss caused by prediction errors. For MAX-k-SAT with ε prediction errors, its global bound gives approximation ratio $1 - O(\varepsilon/n^{\gamma})$ on near-dense instances with $\mathrm{OPT} = \Omega(n^{k-1+\gamma})$, and its local bound gives $1 - O(\Delta\varepsilon/n)$ when each literal occurs in at most Δ clauses; both extend beyond the dense regime (sections 3 and 5).

3. *PAC-style guarantees for oracle selection.* We bound the pseudo-dimension of the cost class induced by LEAP, which gives a sample-complexity guarantee for ERM-based oracle selection and an end-to-end bound that separates the footprint loss of the best oracle from the statistical error (section 4).

4. *Empirical evaluation.* We evaluate LEAP on MAX-CUT and MAX-SAT instances. The experiments show that LEAP reduces the objective gap of imperfect predictions and that selection by empirical induced cost can identify useful oracle candidates. We also compare LEAP with state-of-the-art solvers on large MAX-SAT application benchmarks (section 6).

1.3 Roadmap

We discuss related work in section 2. Section 3 presents LEAP and its error-footprint analysis. Section 4 establishes the PAC-style guarantee for oracle selection. Section 5 applies the framework to MAX-CUT and MAX-k-SAT. Section 6 evaluates LEAP on MAX-CUT and MAX-SAT instances. Section 7 concludes the paper. The appendix contains supporting material, including extended related work, additional experimental details, omitted proofs, and the smoothness verification for MAX-k-CSP, to which MAX-SNP problems reduce.

2 Related Work

We position the paper against three lines of work: smooth integer programming and dense CSP approximation, learning-augmented approximation algorithms, and PAC-style learning of algorithmic decisions.

Smooth Integer Programs and Prediction-Guided Relaxations Smooth integer programs were introduced by Arora et al. (1999) as a polynomial framework for dense constraint satisfaction and related MAX-SNP problems. Fotakis et al. (2016) extended this sampling approach to the near-dense scale $\Omega(n^{k-1+\gamma})$, called almost-sparse in their work. More recently, Bampis et al. (2024) incorporated predictions into the sampling-and-LP framework through a parsimonious oracle. Following the same intuition, our framework uses the oracle prediction as a reference point for constructing the local quantities in the relaxation. The main technical question is how prediction errors affect the relaxation and the rounded solution. Our analysis answers this question by charging prediction errors to the components of the decomposed objective that they can influence, yielding both global smoothness consequences and local consequences when the instance structure permits them.

Learning-Augmented Approximation Algorithms For approximation problems, prior work on learning-augmented algorithms has studied several advice models. In label-advice models, the algorithm receives labels that are correct with probability slightly above $1/2$, often under independence or pairwise-independence assumptions (Cohen-Addad et al., 2024; Ghoshal et al., 2025). In subset-advice models, the optimal labels of a random subset of variables are revealed (Cohen-Addad et al., 2024; Attias et al., 2025). In edge-advice models for graph problems, each edge (i, j) provides local predictions about x_i^* and x_j^* (Aamand et al., 2025). These models yield prediction-sensitive guarantees for MAX-CUT, MAX-SAT, and related CSPs. Our framework studies smooth integer programs with a full oracle prediction $\widehat{\mathbf{x}}$. The guarantee does not impose independence assumptions on oracle errors; its loss is measured through $\|\widehat{\mathbf{x}} - \mathbf{x}^*\|_1$ and the footprint budget. A more detailed comparison is given in section A.1.

Learning Algorithmic Decisions from Data A complementary line of work studies how to learn algorithmic decisions from instance distributions. Gupta and Roughgarden (2017) introduced a PAC framework for algorithm selection, where empirical performance over sampled instances is used to choose from a family of algorithms. The central statistical requirement is a bounded pseudo-dimension for the induced real-valued cost class, which gives uniform convergence from empirical cost to expected cost. This perspective has been developed further for branch-and-bound, algorithm configuration, and related families of algorithmic decisions (Balcan et al., 2018, 2017, 2021; Khodak et al., 2022; Bartlett et al., 2022; Balcan et al., 2024). We use this perspective to study oracle

Symbol	Meaning
$[n]$	Index set $\{1, \ldots, n\}$.
$a \pm b$	Closed interval $[a - b, a + b]$.
n, d	Number of Boolean variables and polynomial degree.
$O(\cdot), \Omega(\cdot)$	Asymptotic notation in n; hidden factors depend only on the fixed degree d.
$\mathbf{x}$	A generic Boolean vector in $\{0,1\}^n$.
$\mathbf{x}^*$	An optimal assignment for the instance.
$\widehat{\mathbf{x}}$	Oracle prediction used as the reference assignment.
ε	Prediction error $\|\widehat{\mathbf{x}} - \mathbf{x}^*\|_1$, or an error budget in the relaxation.
$\mathbf{y}, \mathbf{z}$	Fractional LP solution and rounded integral assignment.
OPT	Optimal objective value $p(\mathbf{x}^*)$.
p	Smooth polynomial objective to be maximized over $\{0,1\}^n$.
β	Smoothness parameter of the polynomial objective.
$\Phi_\varepsilon(\widehat{\mathbf{x}})$	Footprint budget around the prediction.
B	Upper bound on the local coordinate load in the footprint analysis.
Π, π	Instance space and a sampled optimization instance.
$\mathcal{F}, f$	Predictor class and a predictor $f : \Pi \to \{0,1\}^n$.
$\mathcal{A}_f$	LEAP algorithm induced by predictor f.

Table 1: Notation used throughout the paper.

selection. Each predictor f induces a LEAP algorithm, and ERM selects a predictor by the cost of the induced algorithm. Combining the pseudo-dimension bound with the error-footprint guarantee gives an end-to-end learnability guarantee for learning-augmented smooth integer programs.

3 Approach

This section develops the formal construction of LEAP and proves its approximation guarantee, addressing Q1. We first recall smooth Boolean polynomial objectives and their recursive decompositions. We then study quadratic objectives as a motivating special case, where the surrogate objective and the footprint budget have their simplest form. We use this case to formulate the general footprint relaxation for fixed-degree smooth objectives, prove its approximation guarantee, and derive global and local bounds on the footprint budget. The notation used throughout the paper is summarized in table 1.

3.1 Preliminaries

We recall the smooth polynomial model and the recursive decomposition used to construct the relaxation.

Definition 3.1 (β-smooth)**.** An n-variate polynomial $p(\mathbf{x})$ of degree d is *β-smooth* if there exists a constant $\beta > 0$ such that for every $0 \le \ell \le d$, the absolute value of the coefficient of any degree-ℓ monomial in the expansion of $p(\mathbf{x})$ is bounded by $\beta n^{d-\ell}$.

Many fundamental optimization problems have smooth polynomial objective formulations. We recall two examples used throughout the paper.

Example 1. Given an undirected graph $G = (V, E)$, MAX-CUT partitions V to maximize the number of crossing edges. We define the objective over $\mathbf{x} \in \{0,1\}^n$, with $x_i = 1$ denoting that vertex i is in the subset, as

$$p(\mathbf{x}) = \sum_{\{i,j\} \in E} (x_i(1 - x_j) + x_j(1 - x_i)),$$

which is 2-smooth.

Example 2. Given a formula in conjunctive normal form without duplicate clauses, where each clause has exactly k literals on distinct variables, MAX-k-SAT maximizes the number of satisfied clauses. The objective function is

$$p(\mathbf{x}) = \sum_{C \in \mathcal{C}} \left(1 - \prod_{i \in C^-} x_i \prod_{i \in C^+} (1 - x_i) \right),$$

where C^+ and C^- denote indices of positive and negative literals in clause C. This degree-k polynomial is 4^k-smooth.

Further smoothness verification for MAX-k-CSP is given in section E.

The main algebraic tool is a recursive decomposition of p. The root is indexed by $\emptyset$, with $p_\emptyset = p$. For a node I, its children are indexed by (I, j), $j \in [n]$.

Lemma 3.1. *Let $p(\mathbf{x})$ be an n-variate β-smooth polynomial of degree d. For every node I whose component p_I has positive degree, there exist a constant c_I and child components $p_{I,j}$ such that*

$$p_I(\mathbf{x}) = c_I + \sum_{j \in [n]} x_j p_{I,j}(\mathbf{x}).$$

Each nonzero child component has smaller degree and remains β-smooth.

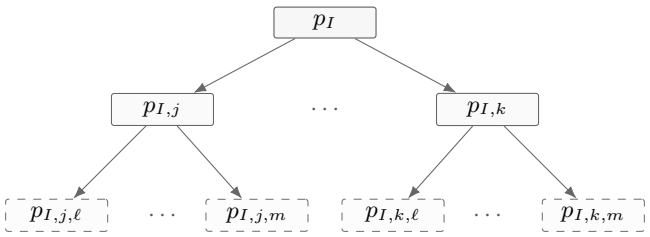


Figure 1: A schematic view of the recursive decomposition. Each node represents a component p_I.

We will use the following global magnitude bound for smooth polynomials.

Lemma 3.2 (Lemma 3.2, Arora et al. 1999)**.** *Let $p(\mathbf{x})$ be an n-variate β-smooth polynomial of degree d. If $n > d$, then for any $\mathbf{x} \in [0,1]^n$, it holds that $|p(\mathbf{x})| \leq 2\,\beta e\, n^d$.*

3.2 A Quadratic Warm-Up

The error-footprint construction is simplest for quadratic objectives. Let p be an n-variate degree-2 β-smooth polynomial. Using lemma 3.1, write

$$p(\mathbf{x}) = c + \sum_{i \in [n]} x_i p_i(\mathbf{x}), \qquad p_i(\mathbf{x}) = c_i + \sum_{j \in [n]} x_j p_{i,j},$$

where $p_{i,j}$ is a constant coefficient. Given an oracle prediction $\widehat{\mathbf{x}}$, define the linear surrogate

$$q(\mathbf{x}) = c + \sum_{i\in[n]} x_i p_i(\widehat{\mathbf{x}}).$$

We introduce $\mathbf{u} \in [0,1]^n$ as a fractional indicator of the mispredicted coordinates. If coordinate j is mispredicted, it can change the local coefficient p_i by at most $|p_{i,j}|$. Thus the tolerance for the i-th local coefficient is

$$\tau_i(\mathbf{u}) = \sum_{j\in[n]} |p_{i,j}| u_j.$$

The total amount charged to coordinate j is its footprint weight

$$F_j = \sum_{i\in[n]} |p_{i,j}|,$$

and for an error size ε, the quadratic footprint budget is

$$\Phi_\varepsilon^{(2)} = \max_{\substack{\mathbf{u}\in[0,1]^n \\ \sum_j u_j \le \varepsilon}} \sum_{j\in[n]} F_j u_j.$$

This is the largest decomposed objective mass that can be affected by ε prediction errors around $\widehat{\mathbf{x}}$.

The corresponding footprint relaxation uses the surrogate objective q and requires each local coefficient to match its predicted value up to the certificate tolerance:

$$\begin{aligned} \max_{\mathbf{y},\mathbf{u}} \quad & c + \sum_{i\in[n]} y_i p_i(\widehat{\mathbf{x}}) \\ \text{s.t.} \quad & p_i(\mathbf{y}) \in p_i(\widehat{\mathbf{x}}) \pm \tau_i(\mathbf{u}) \quad \forall i \in [n], \\ & \sum_{j\in[n]} u_j \le \varepsilon, \\ & \mathbf{y} \in [0,1]^n, \quad \mathbf{u} \in [0,1]^n. \end{aligned} \tag{2-LP-F}$$

For every feasible $(\mathbf{y}, \mathbf{u})$,

$$\begin{aligned} |p(\mathbf{y}) - q(\mathbf{y})| &\le \sum_{i\in[n]} y_i |p_i(\mathbf{y}) - p_i(\widehat{\mathbf{x}})| \le \sum_{i\in[n]} \tau_i(\mathbf{u}) \\ &= \sum_{j\in[n]} F_j u_j \le \Phi_\varepsilon^{(2)}. \end{aligned}$$

Let $S = \{j : \widehat{x}_j \neq x_j^*\}$, where $\mathbf{x}^*$ is an optimal assignment, and set $u_j^* = \mathbf{1}[j \in S]$. When $|S| = \varepsilon$, the pair $(\mathbf{x}^*, \mathbf{u}^*)$ is feasible for (2-LP-F). If $(\mathbf{y}, \mathbf{u})$ is an optimal LP solution, then LP optimality gives

$$p(\mathbf{y}) \ge q(\mathbf{y}) - \Phi_\varepsilon^{(2)} \ge q(\mathbf{x}^*) - \Phi_\varepsilon^{(2)} \ge p(\mathbf{x}^*) - 2\Phi_\varepsilon^{(2)}.$$

The general construction below applies the same bookkeeping to the full decomposition tree.

3.3 General Footprint Relaxation

We consider an n-variate degree-d, β-smooth polynomial objective:

$$\begin{aligned} \max_{\mathbf{x}} \quad & p(\mathbf{x}) \\ \text{s.t.} \quad & \mathbf{x} \in \{0,1\}^n. \end{aligned} \tag{d-IP}$$

Fix a recursive decomposition tree of p as in lemma 3.1. We write $\mathcal{T}$ for the set of generated nodes and use $\emptyset$ for the root. The relaxation constrains the non-root nodes whose components have positive degree:

$$\mathcal{I} = \{I \in \mathcal{T} : I \neq \emptyset,\ \deg(p_I) > 0\}.$$

For every component p_I in the decomposition tree, define its one-step linear surrogate at $\widehat{\mathbf{x}}$ by

$$q_I(\mathbf{y}) = c_I + \sum_{j\in[n]} y_j p_{I,j}(\widehat{\mathbf{x}}).$$

For $\mathbf{u} \in [0,1]^n$, define the tolerance for component I as

$$\tau_I(\mathbf{u}) = \sum_{j\in[n]} |p_{I,j}(\widehat{\mathbf{x}})| u_j.$$

The recursive footprint weight of coordinate j is

$$F_j(\widehat{\mathbf{x}}) = \sum_{I\in\mathcal{I}} |p_{I,j}(\widehat{\mathbf{x}})|,$$

and the footprint budget is

$$\Phi_\varepsilon(\widehat{\mathbf{x}}) = \max_{\substack{\mathbf{u}\in[0,1]^n \\ \sum_j u_j \le \varepsilon}} \sum_{j\in[n]} F_j(\widehat{\mathbf{x}}) u_j.$$

The footprint relaxation is

$$\begin{aligned} \max_{\mathbf{y},\mathbf{u}} \quad & c_\emptyset + \sum_{j\in[n]} y_j p_j(\widehat{\mathbf{x}}) \\ \text{s.t.} \quad & q_I(\mathbf{y}) \in p_I(\widehat{\mathbf{x}}) \pm \tau_I(\mathbf{u}) \quad \forall I \in \mathcal{I}, \\ & \sum_{j\in[n]} u_j \le \varepsilon, \\ & \mathbf{y} \in [0,1]^n, \quad \mathbf{u} \in [0,1]^n. \end{aligned} \tag{d-LP}$$

Different valid decompositions yield different footprint weights, while the guarantee below applies to any fixed decomposition satisfying lemma 3.1. The construction of (d-LP) is summarized in algorithm 1.

A complete Leap protocol is given in algorithm 2. Since the prediction error is unknown, the algorithm scans all budgets in $\{0,\dots,n\}$ and returns the best rounded candidate.

3.4 Error-Footprint Guarantee

For the main applications of this paper, the polynomial objective is multilinear. In this case, a fractional solution of the footprint relaxation can be rounded deterministically without decreasing the objective value.

Lemma 3.3 (Deterministic rounding for multilinear objectives)**.** *Let $p(\mathbf{x})$ be a multilinear polynomial. For any fractional solution $\mathbf{y} \in [0,1]^n$, the greedy deterministic rounding procedure yields an integral vector $\mathbf{z} \in \{0,1\}^n$ such that*

$$p(\mathbf{z}) \ge p(\mathbf{y}).$$

Algorithm 1: ConstructFootprintLP

Input : Objective p, decomposition tree $\mathcal{T}$, prediction $\widehat{\mathbf{x}}$, error budget ε
Output : Footprint relaxation LP_ε
Set $\mathcal{I} \leftarrow \{I \in \mathcal{T} : I \neq \emptyset,\ \deg(p_I) > 0\}$
Introduce variables $\mathbf{y} \in [0,1]^n$ and $\mathbf{u} \in [0,1]^n$
Set objective $q_\emptyset(\mathbf{y}) \leftarrow c_\emptyset + \sum_{j\in[n]} y_j p_j(\widehat{\mathbf{x}})$
foreach $I \in \mathcal{I}$ **do**
  Set $q_I(\mathbf{y}) \leftarrow c_I + \sum_{j\in[n]} y_j p_{I,j}(\widehat{\mathbf{x}})$
  Set $\tau_I(\mathbf{u}) \leftarrow \sum_{j\in[n]} |p_{I,j}(\widehat{\mathbf{x}})| u_j$
  Add $q_I(\mathbf{y}) \le p_I(\widehat{\mathbf{x}}) + \tau_I(\mathbf{u})$
  Add $q_I(\mathbf{y}) \ge p_I(\widehat{\mathbf{x}}) - \tau_I(\mathbf{u})$
Add $\sum_{j\in[n]} u_j \le \varepsilon$
return LP_ε *with objective* $q_\emptyset$ *and the above constraints*

Algorithm 2: Leap: LEarning-augmented Approximation via linear Programming

Input : Problem instance $\pi \in \Pi$, oracle f, rounding `strategy`
Output : Integer solution $\mathbf{z}^*$
Obtain oracle prediction $\widehat{\mathbf{x}} \leftarrow f(\pi)$
Let p be the objective polynomial of π, and fix a recursive decomposition tree $\mathcal{T}$ for p
for $\varepsilon \in \{0, 1, \ldots, n\}$ **do**
  $\mathsf{LP}_\varepsilon \leftarrow$ ConstructFootprintLP$(p, \mathcal{T}, \widehat{\mathbf{x}}, \varepsilon)$
  Solve LP_ε to obtain fractional solution $(\mathbf{y}, \mathbf{u}^y)$
  if *`strategy` is deterministic* **then**
    Obtain $\mathbf{z}$ using the deterministic greedy strategy
  else
    Obtain $\mathbf{z}$ using randomized rounding
  Yield $\mathbf{z}$ and $p(\mathbf{z})$
return *the candidate* $\mathbf{z}^*$ *with the largest evaluated objective value*

Proof. Round the variables one at a time. When all variables except x_i are fixed, multilinearity makes p affine in x_i. An affine function on $[0,1]$ is maximized at an endpoint, so choosing the better value in $\{0,1\}$ does not decrease the objective. Repeating this for all coordinates gives $p(\mathbf{z}) \ge p(\mathbf{y})$. □

Theorem 3.1 (Error-footprint guarantee)**.** *Let p be a multilinear objective, let $\mathbf{x}^*$ be an optimal assignment, and let $\widehat{\mathbf{x}}$ be the oracle prediction. Set $\varepsilon = \|\widehat{\mathbf{x}} - \mathbf{x}^*\|_1$.* Leap *with deterministic rounding returns an integral assignment $\mathbf{z}$ satisfying*

$$p(\mathbf{z}) \ge OPT - 2\Phi_\varepsilon(\widehat{\mathbf{x}}).$$

Proof. Let $q_\emptyset(\mathbf{y}) = c_\emptyset + \sum_{j\in[n]} y_j p_j(\widehat{\mathbf{x}})$ be the linear objective of the relaxation. For a node I, let $\mathcal{T}(I)$ be the set of non-root constrained descendants of I in the decomposition tree. We first prove, by induction on the remaining degree of p_I, that every feasible $(\mathbf{y}, \mathbf{u})$ satisfies

$$|p_I(\mathbf{y}) - q_I(\mathbf{y})| \le \sum_{J\in\mathcal{T}(I)} \tau_J(\mathbf{u}).$$

If p_I is linear, then every nonzero child component is constant, so $p_I(\mathbf{y}) = q_I(\mathbf{y})$. For the induction step,

$$p_I(\mathbf{y}) - q_I(\mathbf{y}) = \sum_{j\in[n]} y_j(p_{I,j}(\mathbf{y}) - p_{I,j}(\widehat{\mathbf{x}})).$$

For each child, we add and subtract its surrogate $q_{I,j}(\mathbf{y})$. The feasibility constraint gives

$$|q_{I,j}(\mathbf{y}) - p_{I,j}(\widehat{\mathbf{x}})| \le \tau_{I,j}(\mathbf{u}),$$

and the induction hypothesis bounds $|p_{I,j}(\mathbf{y}) - q_{I,j}(\mathbf{y})|$ by the tolerances in the child subtree. Summing over children gives exactly the tolerances over $\mathcal{T}(I)$.

Applying this recurrence at the root, where $\mathcal{T}(\emptyset) = \mathcal{I}$, yields

$$|p(\mathbf{y}) - q_\emptyset(\mathbf{y})| \le \sum_{I\in\mathcal{I}} \tau_I(\mathbf{u}) = \sum_{j\in[n]} F_j(\widehat{\mathbf{x}})u_j.$$

In particular, if $\sum_j u_j \le \varepsilon$, then

$$|p(\mathbf{y}) - q_\emptyset(\mathbf{y})| \le \Phi_\varepsilon(\widehat{\mathbf{x}}).$$

Let $S = \{j : \widehat{x}_j \ne x^*_j\}$, and set $u^*_j = \mathbf{1}[j \in S]$. For every $I \in \mathcal{I}$,

$$|q_I(\mathbf{x}^*) - p_I(\widehat{\mathbf{x}})| = |q_I(\mathbf{x}^*) - q_I(\widehat{\mathbf{x}})| \le \sum_{j\in S} |p_{I,j}(\widehat{\mathbf{x}})| = \tau_I(\mathbf{u}^*).$$

Thus $(\mathbf{x}^*, \mathbf{u}^*)$ is feasible for the relaxation with budget $\varepsilon = |S|$. Let $(\mathbf{y}, \mathbf{u}^y)$ be an optimal solution of that relaxation. By LP optimality,

$$q_\emptyset(\mathbf{y}) \ge q_\emptyset(\mathbf{x}^*).$$

Combining these inequalities gives

$$p(\mathbf{y}) \ge q_\emptyset(\mathbf{y}) - \Phi_\varepsilon(\widehat{\mathbf{x}}) \ge q_\emptyset(\mathbf{x}^*) - \Phi_\varepsilon(\widehat{\mathbf{x}}) \ge p(\mathbf{x}^*) - 2\Phi_\varepsilon(\widehat{\mathbf{x}}).$$

Finally, lemma 3.3 gives $p(\mathbf{z}) \ge p(\mathbf{y})$. Since LEAP returns the best evaluated candidate over all enumerated error budgets, its returned assignment also satisfies the same bound. □

For non-multilinear smooth objectives, deterministic rounding may not preserve the objective value. The same footprint argument can be combined with independent randomized rounding.

Lemma 3.4 (Randomized rounding for smooth objectives)**.** *Let $\mathbf{y} \in [0,1]^n$, and let $\mathbf{z} \in \{0,1\}^n$ be generated via independent randomized rounding where $\Pr[z_i = 1] = y_i$ for all $i \in [n]$. Consider an n-variate degree-d polynomial $p(\mathbf{x})$ that is β-smooth. For any $s > d$, with probability at least $1 - 2d/n^{s-d+2}$,*

$$|p(\mathbf{y}) - p(\mathbf{z})| \le \eta_d \beta n^{d-1}\sqrt{\frac{s+1}{2}}\sqrt{n\ln n},$$

where η_d is a constant depending only on d.

Corollary 3.1 (Smooth randomized guarantee)**.** *Let p be an n-variate degree-d, β-smooth objective, let $\mathbf{x}^*$ be an optimal assignment, and set $\varepsilon = \|\widehat{\mathbf{x}} - \mathbf{x}^*\|_1$. Under randomized rounding, for any $s > d$, with probability at least $1 - 2d/n^{s-d+2}$,* LEAP *returns an assignment $\mathbf{z}$ satisfying*

$$p(\mathbf{z}) \ge OPT - 2\Phi_\varepsilon(\widehat{\mathbf{x}}) - \eta_d \beta n^{d-1}\sqrt{\frac{s+1}{2}}\sqrt{n\ln n}.$$

Enumerating the error budget. The proof of theorem 3.1 uses only one budget that covers the true error, and a budget within a factor of two of the error suffices. The full scan in algorithm 2 can therefore be replaced by the doubling grid

$$\mathcal{B} = \{0, n\} \cup \{2^j : j \in \mathbb{Z}_{\geq 0},\ 2^j < n\},$$

which has $O(\log n)$ elements.

Proposition 3.1 (Doubling budget grid)**.** *Under the conditions of theorem 3.1,* Leap *run over the budgets in $\mathcal{B}$ solves $O(\log n)$ linear programs and returns an assignment $\mathbf{z}$ satisfying*

$$p(\mathbf{z}) \geq OPT - 4\Phi_\varepsilon(\widehat{\mathbf{x}}).$$

Under the conditions of corollary 3.1, the guarantee there holds with $2\Phi_\varepsilon(\widehat{\mathbf{x}})$ replaced by $4\Phi_\varepsilon(\widehat{\mathbf{x}})$, and with the same rounding term and success probability.

Proof. If $\varepsilon = 0$, the budget $0 \in \mathcal{B}$ recovers theorem 3.1. Otherwise, $\mathcal{B}$ contains a budget b with $\varepsilon \leq b < 2\varepsilon$. Since $b \geq \varepsilon$, the certificate $(\mathbf{x}^*, \mathbf{u}^*)$ remains feasible, and the proof of theorem 3.1 gives $p(\mathbf{z}) \geq \mathrm{OPT} - 2\Phi_b(\widehat{\mathbf{x}})$. Scaling a maximizer of $\Phi_b(\widehat{\mathbf{x}})$ by ε/b gives a feasible vector for $\Phi_\varepsilon(\widehat{\mathbf{x}})$, so $\Phi_b(\widehat{\mathbf{x}}) \leq (b/\varepsilon)\Phi_\varepsilon(\widehat{\mathbf{x}}) \leq 2\Phi_\varepsilon(\widehat{\mathbf{x}})$. The randomized case follows by applying lemma 3.4 at budget b. □

We state the remaining results for the full scan; each extends to the doubling grid with $4\Phi_\varepsilon(\widehat{\mathbf{x}})$ in place of $2\Phi_\varepsilon(\widehat{\mathbf{x}})$.

3.5 Analyzing the Footprint Budget

The preceding guarantees depend on the instance-dependent quantity $\Phi_\varepsilon(\widehat{\mathbf{x}})$. We first bound this quantity using only global smoothness, and then give a local refinement for decompositions that preserve variable support.

Lemma 3.5 (Global footprint bound)**.** *Let $p(\mathbf{x})$ be an n-variate degree-d, β-smooth polynomial with $d \geq 2$ and $n > d$. There is a constant κ_d depending only on d such that*

$$\Phi_\varepsilon(\widehat{\mathbf{x}}) \leq \kappa_d \beta n^{d-1} \varepsilon.$$

Proof. Let $\mathcal{I}_r \subseteq \mathcal{I}$ be the set of decomposition tuples of length r. The root objective is not a component constraint, so the relevant levels are $r = 1, \ldots, d-1$. Fix any $\mathbf{u} \in [0,1]^n$ satisfying $\sum_j u_j \leq \varepsilon$.

For $1 \leq r < d-1$, each child component $p_{I,j}$ with $I \in \mathcal{I}_r$ has degree at most $d-r-1$ and remains β-smooth. By lemma 3.2,

$$|p_{I,j}(\widehat{\mathbf{x}})| \leq 2e\beta n^{d-r-1}.$$

There are at most n^r tuples in $\mathcal{I}_r$, so the level-r contribution is at most

$$\sum_{I \in \mathcal{I}_r} \sum_{j \in [n]} |p_{I,j}(\widehat{\mathbf{x}})| u_j \leq 2e\beta n^{d-1} \varepsilon.$$

There are $d-2$ such levels. At the final level $r = d-1$, the child components are constant coefficients with magnitude at most β, and therefore

$$\sum_{I \in \mathcal{I}_{d-1}} \sum_{j \in [n]} |p_{I,j}(\widehat{\mathbf{x}})| u_j \leq \beta n^{d-1} \varepsilon.$$

Summing over all levels gives

$$\sum_{j\in[n]} F_j(\widehat{\mathbf{x}})u_j \le \big(2e(d-2)+1\big)\beta n^{d-1}\varepsilon.$$

Taking the maximum over feasible $\mathbf{u}$ proves the claim for any $\kappa_d \ge 2e(d-2)+1$. □

Corollary 3.2 (Global smoothness consequence)**.** *Let p be a multilinear degree-d, β-smooth objective, let $\mathbf{x}^*$ be an optimal assignment, and set $\varepsilon = \|\widehat{\mathbf{x}} - \mathbf{x}^*\|_1$. Under deterministic rounding,* Leap *returns an assignment $\mathbf{z}$ satisfying*

$$p(\mathbf{z}) \ge OPT - 2\kappa_d\beta n^{d-1}\varepsilon.$$

Proof. Combine theorem 3.1 with lemma 3.5. □

The global bound ignores which variables appear in each component. To capture local structure, fix a recursive decomposition and write $\mathrm{supp}(p_I)$ for the variables appearing in component p_I. The decomposition is *support-preserving* if every child remains within the support of its parent: $p_{I,j} \equiv 0$ whenever $j \notin \mathrm{supp}(p_I)$, and $\mathrm{supp}(p_{I,j}) \subseteq \mathrm{supp}(p_I)\setminus\{j\}$ otherwise. We say this decomposition has coordinate load at most B at $\widehat{\mathbf{x}}$ if

$$\sum_{I\in\mathcal{I}:\ j\in\mathrm{supp}(p_I)} |p_{I,j}(\widehat{\mathbf{x}})| \le B \quad \text{for every } j\in[n].$$

Lemma 3.6 (Coordinate-load footprint bound)**.** *Fix a support-preserving recursive decomposition used to construct* (d-LP)*. If the decomposition has coordinate load at most B at $\widehat{\mathbf{x}}$, then*

$$\Phi_\varepsilon(\widehat{\mathbf{x}}) \le B\varepsilon.$$

Proof. Let $F_{(1)}(\widehat{\mathbf{x}}) \ge \cdots \ge F_{(n)}(\widehat{\mathbf{x}})$ be the footprint weights in decreasing order. The optimization defining $\Phi_\varepsilon(\widehat{\mathbf{x}})$ is a fractional knapsack problem with unit costs:

$$\Phi_\varepsilon(\widehat{\mathbf{x}}) = \max_{\substack{\mathbf{u}\in[0,1]^n \\ \sum_j u_j \le \varepsilon}} \sum_{j\in[n]} F_j(\widehat{\mathbf{x}})u_j.$$

Since ε is integral, an optimal solution places all budget on the ε largest weights, so

$$\Phi_\varepsilon(\widehat{\mathbf{x}}) = \sum_{t=1}^{\varepsilon} F_{(t)}(\widehat{\mathbf{x}}).$$

Under a support-preserving decomposition, every nonzero contribution to $F_j(\widehat{\mathbf{x}})$ comes from a component whose support contains j. The coordinate-load condition therefore gives $F_j(\widehat{\mathbf{x}}) \le B$ for every j, and hence $\Phi_\varepsilon(\widehat{\mathbf{x}}) \le B\varepsilon$. □

Corollary 3.3 (Local footprint consequence)**.** *Let p be a multilinear objective, and suppose the chosen support-preserving decomposition has coordinate load at most B at $\widehat{\mathbf{x}}$. Set $\varepsilon = \|\widehat{\mathbf{x}} - \mathbf{x}^*\|_1$. Under deterministic rounding,* Leap *returns an assignment $\mathbf{z}$ satisfying*

$$p(\mathbf{z}) \ge OPT - 2B\varepsilon.$$

Proof. Combine theorem 3.1 with lemma 3.6. □

These additive bounds can be converted into multiplicative ratios when a lower bound on OPT is available. We carry out this conversion for MAX-CUT and MAX-k-SAT in section 5.

3.6 Complexity and Properties

For fixed degree d, the decomposition contributes $O(n^{d-1})$ tuple-indexed constraints and polynomially many variables, so each relaxation has polynomial description size. Let N_d denote this description size and $T_{\mathtt{LP}}$ the running time of the chosen polynomial-time LP solver. Since the LP solves dominate the running time, full budget enumeration takes $O(n \cdot T_{\mathtt{LP}}(N_d))$ time, and the doubling grid of proposition 3.1 reduces this to $O(\log n \cdot T_{\mathtt{LP}}(N_d))$.

With a perfect prediction, the footprint budget is zero, and for multilinear objectives deterministic rounding returns an optimal solution.

4 Learnability of Oracles

Section 3 analyzes LEAP for a fixed predictor. This section studies how to select a predictor from sampled optimization instances. Following the PAC perspective of Gupta and Roughgarden (2017), we view each predictor f as inducing an algorithm $\mathcal{A}_f$: its prediction is fed into the footprint relaxation and the subsequent rounding procedure.

4.1 Oracle-Induced Algorithm Classes

Let Π denote the instance space. Each instance $\pi \in \Pi$ is a smooth Boolean polynomial optimization problem of the form (d-IP); write p_π for its objective. We assume that all instances in the family have the same fixed size and that all objective values lie in a common range $[0, H]$ with $H > 0$. Let $\mathcal{D}$ be an unknown but fixed distribution over Π.

Let $\mathcal{F}$ be a hypothesis class of predictors $f : \Pi \to \{0,1\}^n$, viewed as candidate oracles. For $f \in \mathcal{F}$ and instance π, LEAP uses $\widehat{\mathbf{x}} = f(\pi)$ and returns an integral assignment $\mathbf{z}_f(\pi)$; unless stated otherwise, LEAP refers to algorithm 2 with the full budget scan. Thus f induces a LEAP algorithm $\mathcal{A}_f$, and we write

$$\mathcal{A}_{\mathcal{F}} = \{\mathcal{A}_f : f \in \mathcal{F}\}$$

for the induced algorithm class.

Let $\mathbf{x}^*(\pi)$ denote an optimal assignment for instance π. The expected value achieved by predictor f is

$$\mathcal{P}(f) \triangleq \mathbb{E}_{\pi\sim\mathcal{D}}\big[p_\pi(\mathbf{z}_f(\pi))\big],$$

and the expected optimum is

$$\mathcal{P}^* \triangleq \mathbb{E}_{\pi\sim\mathcal{D}}\big[p_\pi(\mathbf{x}^*(\pi))\big].$$

When randomized rounding is used, the expectation defining $\mathcal{P}(f)$ also includes the algorithm's internal randomness.

4.2 Empirical Cost Minimization

We select predictors according to the performance of the algorithms they induce. Since objective values lie in $[0, H]$, define the cost of $\mathcal{A}_f$ on π as

$$\mathtt{COST}(\mathcal{A}_f, \pi) \triangleq H - p_\pi(\mathbf{z}_f(\pi)).$$

For example, in a MAX-k-SAT family with a fixed number of clauses, H may be taken as the number of clauses, and the cost is the number of clauses not satisfied by the returned assignment.

The expected risk is

$$R(f) \triangleq \mathbb{E}_{\pi\sim\mathcal{D}}\big[\mathtt{COST}(\mathcal{A}_f, \pi)\big],$$

and let

$$f^*_{\mathtt{cost}} \in \arg\min_{f \in \mathcal{F}} R(f)$$

be a cost-optimal oracle in the hypothesis class. Since $\mathtt{COST}(\mathcal{A}_f, \pi) = H - p_\pi(\mathbf{z}_f(\pi))$, we have $R(f) = H - \mathcal{P}(f)$.

Given an i.i.d. sample $S = \{\pi_1, \ldots, \pi_m\}$ from $\mathcal{D}$, empirical risk minimization (ERM) selects

$$\hat{f}_{\mathrm{ERM}} \in \arg\min_{f \in \mathcal{F}} \frac{1}{m} \sum_{i=1}^{m} \mathtt{COST}(\mathcal{A}_f, \pi_i).$$

4.3 Pseudo-Dimension Bound

We bound the complexity of the induced cost class. Since the cost is real-valued, the relevant notion is pseudo-dimension, the standard real-valued analogue of VC dimension (Pollard, 1990; Anthony and Bartlett, 1999). For each coordinate $i \in [n]$, let $\mathcal{F}_i$ be the class of i-th coordinate predictors, and define

$$d_{\mathcal{F}} \triangleq \sum_{i=1}^{n} \mathrm{VCdim}(\mathcal{F}_i).$$

We assume $d_{\mathcal{F}} \geq 1$.

Lemma 4.1. *Let $\mathcal{F}$ be a hypothesis class of predictors $f : \Pi \to \{0,1\}^n$, and let $\mathcal{A}_{\mathcal{F}}$ be the induced class of* LEAP *algorithms. The pseudo-dimension of the cost functions*

$$\{\pi \mapsto \mathtt{COST}(\mathcal{A}_f, \pi) : f \in \mathcal{F}\}$$

satisfies

$$\mathrm{Pdim}(\mathcal{A}_{\mathcal{F}}) \leq C\, d_{\mathcal{F}} \log(e\, d_{\mathcal{F}})$$

for some absolute constant C.

Proof. Throughout, deterministic tie-breaking is fixed for the LP solver, the rounding procedure, and final candidate selection. If the algorithm uses internal randomness, we include the random seed in the input domain. Let Ξ denote the seed space and define

$$h_f : \Pi \times \Xi \to [0, H], \qquad h_f(\pi, \xi) = \mathtt{COST}(\mathcal{A}_f(\pi; \xi)).$$

It suffices to bound the pseudo-dimension of $\mathcal{H} = \{h_f : f \in \mathcal{F}\}$.

Fix a finite set $S = \{(\pi_1, \xi_1), \ldots, (\pi_m, \xi_m)\} \subseteq \Pi \times \Xi$ and thresholds $r_1, \ldots, r_m \in \mathbb{R}$. For each $f \in \mathcal{F}$, define a labeling vector

$$\ell_f(j) = \mathbb{I}\{h_f(\pi_j, \xi_j) > r_j\}.$$

For fixed (π_j, ξ_j), the final cost is determined by the prediction $f(\pi_j)$, since the instance and seed fix the subsequent relaxation and rounding outcomes. Thus the number of possible labelings $\{\ell_f\}$ is at most the number of distinct prediction matrices

$$P_f \in \{0,1\}^{n \times m}, \qquad [P_f]_{i,j} = f_i(\pi_j).$$

Let $d_i = \mathrm{VCdim}(\mathcal{F}_i)$, and assume $m \geq 1$. If $d_i = 0$, the coordinate class yields at most one distinct vector on the sample. For $1 \leq d_i \leq m$, Sauer's lemma (Sauer, 1972) bounds the number of distinct vectors $(f_i(\pi_1), \ldots, f_i(\pi_m))$ by

$$\sum_{s=0}^{d_i} \binom{m}{s} \leq \left(\frac{em}{d_i}\right)^{d_i} \leq (em)^{d_i}.$$

If $d_i > m$, the trivial bound $2^m \le (em)^{d_i}$ applies. Thus $(em)^{d_i}$ bounds the number of coordinate vectors in all cases. Even when the coordinates of f are coupled, the set of valid prediction matrices is a subset of the Cartesian product of the coordinate-wise projections. Hence the number of distinct prediction matrices is at most

$$\prod_{i=1}^{n} (em)^{d_i} = (em)^{d_{\mathcal{F}}}.$$

If S is pseudo-shattered by $\mathcal{H}$, then all 2^m binary labelings must be realized, so

$$2^m \le (em)^{d_{\mathcal{F}}}.$$

This inequality fails for $m > C\, d_{\mathcal{F}} \log(e d_{\mathcal{F}})$ with a sufficiently large absolute constant C, which proves the bound. □

4.4 End-to-End Learnability Guarantee

The pseudo-dimension bound gives a standard ERM guarantee for the induced algorithm class. We combine this guarantee with the footprint bound from section 3 through the following predictor-class quantity. For an instance π, let $\Phi^{\pi}_{\varepsilon}(\widehat{\mathbf{x}})$ denote the footprint budget induced by p_π, $\widehat{\mathbf{x}}$, and ε, and define

$$\Lambda^{\Phi}_{\mathcal{F}} \triangleq 2 \inf_{f \in \mathcal{F}} \mathbb{E}_{\pi \sim \mathcal{D}} \left[\Phi^{\pi}_{\|f(\pi) - \mathbf{x}^*(\pi)\|_1}(f(\pi)) \right].$$

This quantity is the footprint loss of the best predictor in $\mathcal{F}$. The next lemma is where the footprint guarantee enters the learnability analysis.

Lemma 4.2 (Transfer lemma)**.** *If every objective p_π is multilinear and* Leap *uses deterministic rounding, then*

$$\mathcal{P}(f^*_{\texttt{cost}}) \ge \mathcal{P}^* - \Lambda^{\Phi}_{\mathcal{F}}.$$

Proof. By the definition of $f^*_{\texttt{cost}}$, for every $f \in \mathcal{F}$,

$$R(f^*_{\texttt{cost}}) \le R(f).$$

Since $R(f) = H - \mathcal{P}(f)$, this is equivalent to

$$\mathcal{P}(f^*_{\texttt{cost}}) \ge \mathcal{P}(f).$$

For every $f \in \mathcal{F}$ and instance π, theorem 3.1 gives

$$p_\pi(\mathbf{z}_f(\pi)) \ge p_\pi(\mathbf{x}^*(\pi)) - 2\Phi^{\pi}_{\|f(\pi) - \mathbf{x}^*(\pi)\|_1}(f(\pi)).$$

Taking expectations, combining with $\mathcal{P}(f^*_{\texttt{cost}}) \ge \mathcal{P}(f)$, and taking the infimum over $f \in \mathcal{F}$ yields

$$\mathcal{P}(f^*_{\texttt{cost}}) \ge \mathcal{P}^* - \Lambda^{\Phi}_{\mathcal{F}}.$$

□

Combining the ERM excess-risk bound with the transfer lemma gives the final end-to-end guarantee.

Theorem 4.1. *Under the setup above, fix $\alpha > 0$ and $\delta \in (0, 1]$. If*

$$m \geq C \left(\frac{H}{\alpha}\right)^2 \left(d_{\mathcal{F}} \log(e d_{\mathcal{F}}) + \log \frac{1}{\delta}\right),$$

then with probability at least $1-\delta$, any empirical risk minimizer $\hat{f}_{\mathrm{ERM}}$ satisfies $R(\hat{f}_{\mathrm{ERM}}) - R(f^*_{\mathtt{cost}}) \leq 2\alpha$. If, in addition, the conditions of lemma 4.2 hold, then

$$\mathcal{P}(\hat{f}_{\mathrm{ERM}}) \geq \mathcal{P}^* - \Lambda^{\Phi}_{\mathcal{F}} - 2\alpha.$$

Proof. By lemma 4.1, the induced cost class has pseudo-dimension $O(d_{\mathcal{F}} \log(e d_{\mathcal{F}}))$. Applying lemma D.2 with this bound gives the excess-risk statement after adjusting the absolute constant C. Since $R(f) = H - \mathcal{P}(f)$, the excess-risk bound is equivalent to

$$\mathcal{P}(\hat{f}_{\mathrm{ERM}}) \geq \mathcal{P}(f^*_{\mathtt{cost}}) - 2\alpha.$$

Combining this inequality with lemma 4.2 gives the expected optimization guarantee. □

The footprint bounds in sections 3 and 5 turn $\Lambda^{\Phi}_{\mathcal{F}}$ into concrete bounds involving the best expected prediction error

$$\varepsilon_{\mathcal{F}} \triangleq \inf_{f \in \mathcal{F}} \mathbb{E}_{\pi \sim \mathcal{D}} \left[\| f(\pi) - \mathbf{x}^*(\pi) \|_1 \right].$$

The global smoothness bound in corollary 3.2 gives $\Lambda^{\Phi}_{\mathcal{F}} = O(\beta n^{d-1} \varepsilon_{\mathcal{F}})$. For MAX-$k$-SAT with each literal in at most Δ clauses, lemma 5.1 and corollary 3.3 give $\Lambda^{\Phi}_{\mathcal{F}} = O(\Delta \varepsilon_{\mathcal{F}})$.

Doubling budget grid. With the doubling grid of proposition 3.1, lemma 4.2 and theorem 4.1 hold with $\Lambda^{\Phi}_{\mathcal{F}}$ replaced by $2\Lambda^{\Phi}_{\mathcal{F}}$, so the concrete bounds above keep their order. The pseudo-dimension bound uses only that the cost on a fixed instance is determined by the prediction, and proposition 3.1 replaces 2Φ by 4Φ in the transfer lemma.

4.5 Concrete Oracle Families

The statistical guarantee above applies to predictor classes with controlled coordinate-wise VC dimension. This includes bounded-size GNN- and Transformer-based predictors for graph and formula instances (Selsam et al., 2019; Gasse et al., 2019; Pan et al., 2025), which have finite VC dimension (Bartlett et al., 2019). The computational efficiency of this ERM step is a separate question and remains open at this level of generality (Gupta and Roughgarden, 2017; Bartlett et al., 2022).

5 Applications

We instantiate the framework on two examples. The first is MAX-CUT, a quadratic graph cut problem, and the second is MAX-k-SAT, a Boolean constraint satisfaction problem with higher-order interactions. Together, they show how the same LEAP construction applies in both quadratic and higher-degree settings.

5.1 MAX-CUT

Recall the MAX-CUT objective p of example 1, a quadratic, multilinear, 2-smooth polynomial. Since p is multilinear, deterministic rounding applies. If the oracle prediction satisfies $\|\widehat{\mathbf{x}} - \mathbf{x}^*\|_1 \leq \varepsilon$, then corollary 3.2 gives an integral assignment $\mathbf{z}$ such that

$$p(\mathbf{z}) \geq \mathrm{OPT} - C_2 n\varepsilon,$$

where C_2 is an absolute constant for quadratic objectives.

This additive bound gives a relative guarantee whenever OPT is sufficiently large. Suppose that $\mathrm{OPT} = \Omega(n^{1+\gamma})$ for a constant $\gamma \in [0, 1]$. Then

$$\frac{p(\mathbf{z})}{\mathrm{OPT}} \geq 1 - O\left(\frac{\varepsilon}{n^\gamma}\right).$$

This condition is implied by average degree $\Omega(n^\gamma)$, since every graph has a cut of size at least $|E|/2$.

The footprint guarantee also has a local form. A wrong prediction at vertex j can affect only edges incident to j. Applying theorem 3.1 to a support-preserving decomposition of p gives

$$p(\mathbf{z}) \geq \mathrm{OPT} - C \sum_{t=1}^{\varepsilon} d_{(t)},$$

where $d_{(1)} \geq d_{(2)} \geq \cdots$ are the vertex degrees in decreasing order and C is an absolute constant. In particular, if the maximum degree is at most Δ, then

$$p(\mathbf{z}) \geq \mathrm{OPT} - C\Delta\varepsilon.$$

Removing isolated vertices, which do not affect the objective, gives $|E| \geq n/2$ and hence $\mathrm{OPT} \geq |E|/2 \geq n/4$. Thus, for graphs of maximum degree Δ, the approximation ratio is $1 - O(\Delta\varepsilon/n)$.

5.2 MAX-k-SAT

Consider a k-CNF formula on n variables, where each clause has exactly k literals on distinct variables as in example 2, and let $p(\mathbf{x})$ be the number of clauses satisfied by an assignment $\mathbf{x} \in \{0, 1\}^n$. p is a degree-k multilinear polynomial. For formulas without duplicate clauses, the smoothness parameter satisfies $\beta \leq 4^k$. Thus corollary 3.2 gives, for $\|\widehat{\mathbf{x}} - \mathbf{x}^*\|_1 \leq \varepsilon$,

$$p(\mathbf{z}) \geq \mathrm{OPT} - C_k n^{k-1}\varepsilon,$$

where C_k depends only on k.

If $\mathrm{OPT} = \Omega(n^{k-1+\gamma})$ for a constant $\gamma \in [0, 1]$, then

$$\frac{p(\mathbf{z})}{\mathrm{OPT}} \geq 1 - O\left(\frac{\varepsilon}{n^\gamma}\right).$$

The preceding guarantee uses the global smoothness bound and ignores how clauses are distributed across variables. Bounded-occurrence formulas allow a sharper local bound from the same error-footprint guarantee. In this setting, a wrong prediction is charged only to decomposition terms arising from clauses touched by the mispredicted variable. We use a support-preserving decomposition of the multilinear expansion: each monomial is decomposed along a fixed ordering of the variables in its support, so no descendant component introduces variables outside that support.

Lemma 5.1. *Fix a constant k. For a MAX-k-SAT instance in which each literal occurs in at most Δ clauses, the support-preserving decomposition has coordinate load at most $B_k\Delta$ at $\widehat{\mathbf{x}}$, that is,*

$$F_j(\widehat{\mathbf{x}}) \le B_k\Delta \quad \textit{for every } j \in [n],$$

for a constant B_k depending only on k.

Proof. A single MAX-k-SAT clause expands into at most 2^k multilinear monomials, each with coefficient magnitude at most one. Under the support-preserving decomposition, a monomial contributes to the footprint weight $F_j(\widehat{\mathbf{x}})$ only along variables in its support. For a fixed variable j, each monomial containing j can charge j at most once along its decomposition path, and the charged value has magnitude at most the monomial coefficient because $\widehat{\mathbf{x}} \in \{0,1\}^n$. Hence a single clause containing variable j, in either polarity, contributes at most 2^k to $F_j(\widehat{\mathbf{x}})$.

If each literal occurs in at most Δ clauses, then each variable occurs in at most 2Δ clauses. Thus $F_j(\widehat{\mathbf{x}}) \le 2^{k+1}\Delta$ for every j. Taking $B_k = 2^{k+1}$ proves the claim. □

Combining the lemma with corollary 3.3, an oracle prediction satisfying $\|\widehat{\mathbf{x}} - \mathbf{x}^*\|_1 \le \varepsilon$ yields

$$p(\mathbf{z}) \ge \mathrm{OPT} - C'_k\Delta\varepsilon,$$

where C'_k depends only on k. Removing variables that appear in no clause, which do not affect the objective, leaves at least n/k clauses, since each clause contains k variables. A uniformly random assignment satisfies each clause with probability at least $1/2$, so $\mathrm{OPT} \ge n/(2k)$. Consequently,

$$\frac{p(\mathbf{z})}{\mathrm{OPT}} \ge 1 - O\left(\frac{\Delta\varepsilon}{n}\right).$$

When $\Delta = O(1)$, the prediction-dependent additive loss is $O(\varepsilon)$, improving on the global smoothness estimate $O(n^{k-1}\varepsilon)$ in this bounded-occurrence setting.

The same viewpoint also covers MAX-k-CSP, which generalizes MAX-k-SAT to arbitrary Boolean constraints of arity k (section E).

6 Experiments

We study solution quality from imperfect predictions, empirical oracle selection, and a comparison on large application benchmarks.

6.1 Solution Quality from Imperfect Predictions

To control the prediction error exactly, we use *planted* instances, which are generated so that a fixed assignment $\mathbf{x}^*$ cuts every edge for MAX-CUT and satisfies every clause for MAX-SAT, and is therefore optimal. The prediction flips a prescribed fraction of the variables in $\mathbf{x}^*$, so the prediction error equals the number of flipped variables. We compare two uses of this prediction: the direct baseline returns the predicted assignment itself, while Leap passes the prediction through the relaxation-and-rounding pipeline with the full budget scan of algorithm 2. Both methods are scored by the relative objective gap $(\mathrm{OPT} - p(\mathbf{z}))/\mathrm{OPT}$ for $n \in \{100, 200, 500, 1000\}$ and flip fractions from 0% to 20%; construction details are given in section B.1.

Dense instances. This regime tests the global smoothness bound of section 5: with $\mathrm{OPT} = \Theta(n^d)$, it predicts an approximation ratio of $1 - O(\varepsilon/n)$, where ε/n is the fraction of flipped variables.

Figure 2 shows that the gap of the direct prediction grows roughly linearly with the flip fraction, while Leap recovers the planted optimum at every size and flip fraction.

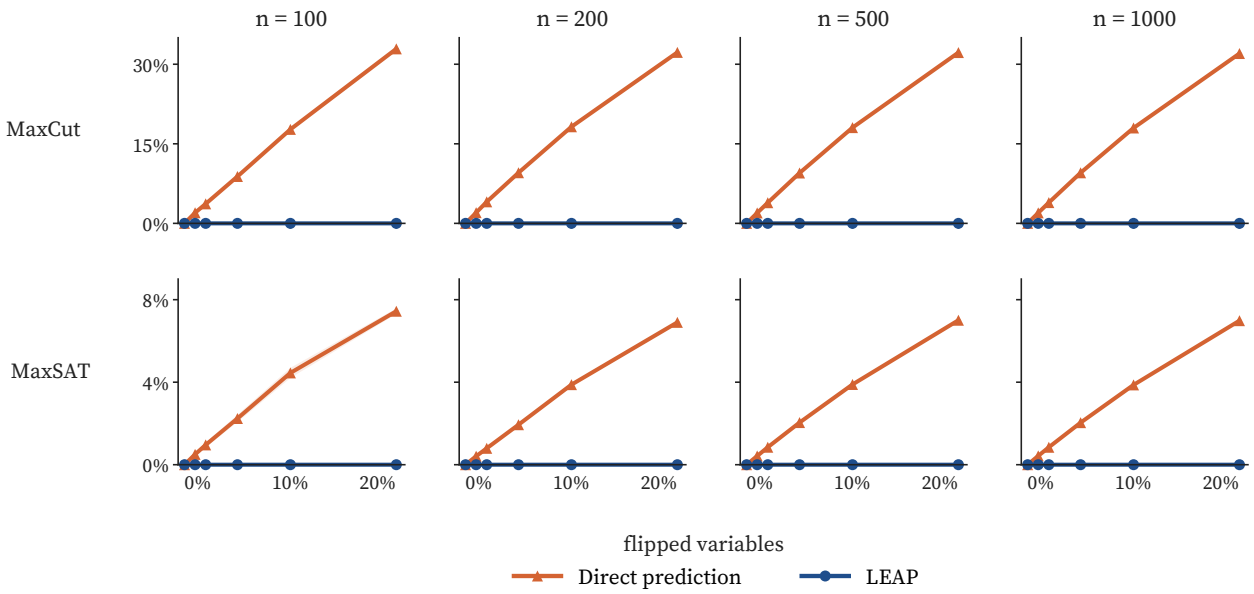


Figure 2: Relative objective gaps versus the fraction of flipped variables for *dense* instances.

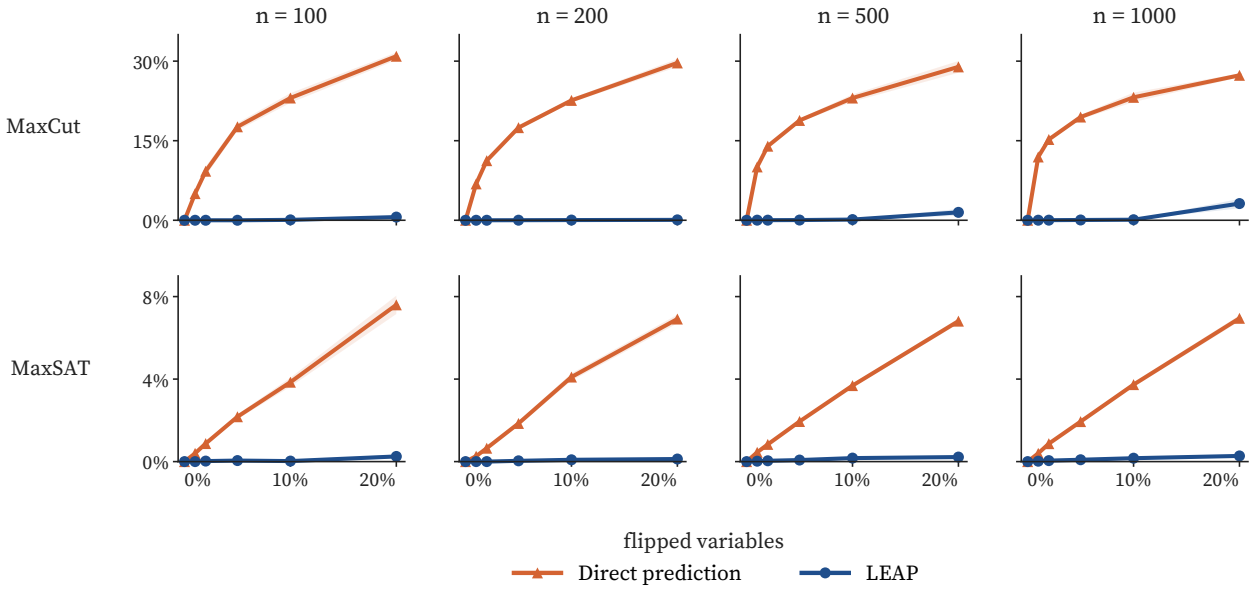


Figure 3: Relative objective gaps versus the fraction of flipped variables for *sparse* instances.

Sparse instances. This regime tests the local coordinate-load bound of corollary 3.3 on MAX-CUT and bounded-occurrence MAX-SAT instances; the bound charges each error only to the edges or clauses it affects.

Figure 3 shows that LEAP reduces the gap of the direct prediction. The reduction persists at the largest flip fraction, with the largest residual gaps on larger MAX-CUT instances.

6.2 Empirical Selection over Oracle Candidates

We next study an empirical version of the learning question in Q2. We consider four finite oracle classes—annealing SLS, WalkSAT-style SLS, ProbSAT-style SLS, and GNN predictors. Within each class, ERM selects the candidate whose induced algorithm has the smallest empirical cost on training instances, and we evaluate the selected candidate on test instances.

We use synthetic MAX-CUT and unweighted MAX-SAT instances with $n = 200$ variables and vary the number m of training instances from 1 to 3000 (see section B for details). For each oracle class $\mathcal{F}$, fig. 4 plots the excess test risk of the ERM-selected candidate $\hat{f}_m$, that is, its average test cost minus the smallest average test cost over $\mathcal{F}$.

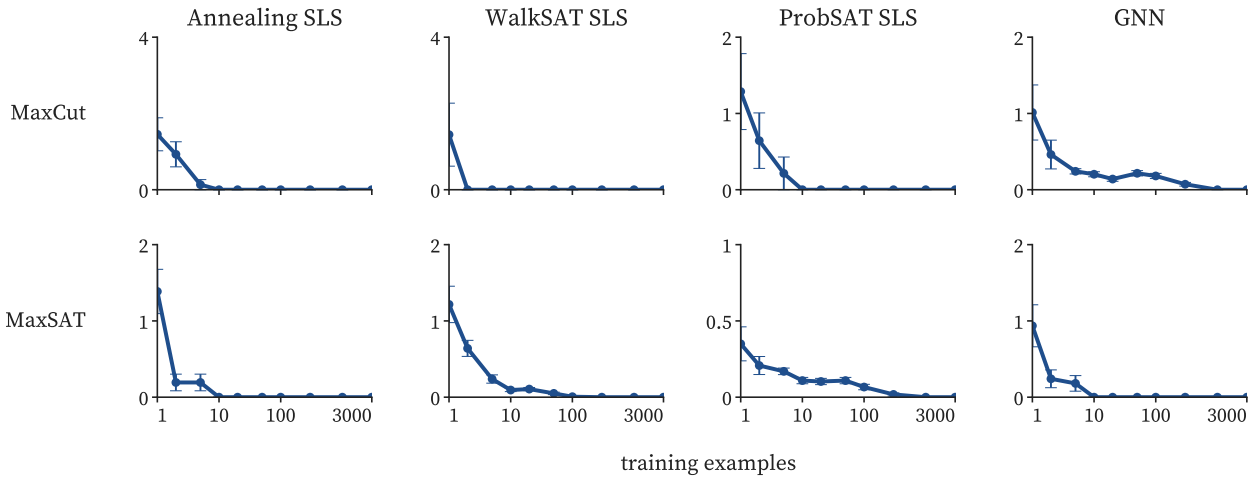


Figure 4: ERM selects useful oracle candidates from training instances. The horizontal axis shows the number of training instances; the vertical axis shows excess test risk, the selected candidate's average test cost minus the smallest average test cost in its oracle class. Error bars show standard errors over repeated draws of the training instances.

Figure 4 shows that the excess test risk decreases steadily as ERM is given more training instances, consistent with theorem 4.1.

6.3 Large MAX-SAT Benchmarks

We finally run Leap on MAX-SAT application benchmarks from eight families and compare with nine state-of-the-art MAX-SAT solver configurations (MaxSAT Evaluation, 2024) on the 256 instances for which all methods return an assignment. Table 2 summarizes the sizes of these instances. Without a learned oracle, Leap starts from a random assignment and runs ten rounds, each using the previous output as its prediction and the doubling grid of proposition 3.1. We report the best of five such runs and one 60-second run per baseline; a single Leap run takes under an hour on most instances, with the longest runs taking tens of hours (see section B.3 for details). The clause-normalized gap is the number of clauses left unsatisfied by Leap minus that of the best observed baseline, as a percentage of the M clauses.

Figure 5 shows that the mean clause-normalized gap is below one percentage point in every family. Together with table 2, this shows that Leap runs on instances with up to about two million clauses while staying close to the best observed baseline.

Table 2: Instance sizes of the eight benchmark families. Entries are mean / maximum over instances; width counts the literals in a clause.

Family	Instances	Variables	Clauses	Width
Deezer	40	25 / 156	169 / 1 684	2.00 / 2
Decision trees	14	27 015 / 82 071	231 206 / 660 735	5.27 / 98
BDD optimization	23	6 491 / 21 190	453 237 / 1 887 499	2.75 / 128
Rule-based classifiers	38	27 611 / 141 265	359 859 / 1 970 958	2.81 / 97
Single-machine scheduling	31	26 343 / 43 672	734 582 / 1 775 621	2.97 / 3
Judgment aggregation	38	12 894 / 22 994	296 276 / 1 567 566	2.89 / 3
Access control	26	1 877 / 2 050	5 043 / 6 900	2.44 / 23
Knowledge-base inconsistency	46	64 919 / 326 096	156 135 / 648 235	2.61 / 121

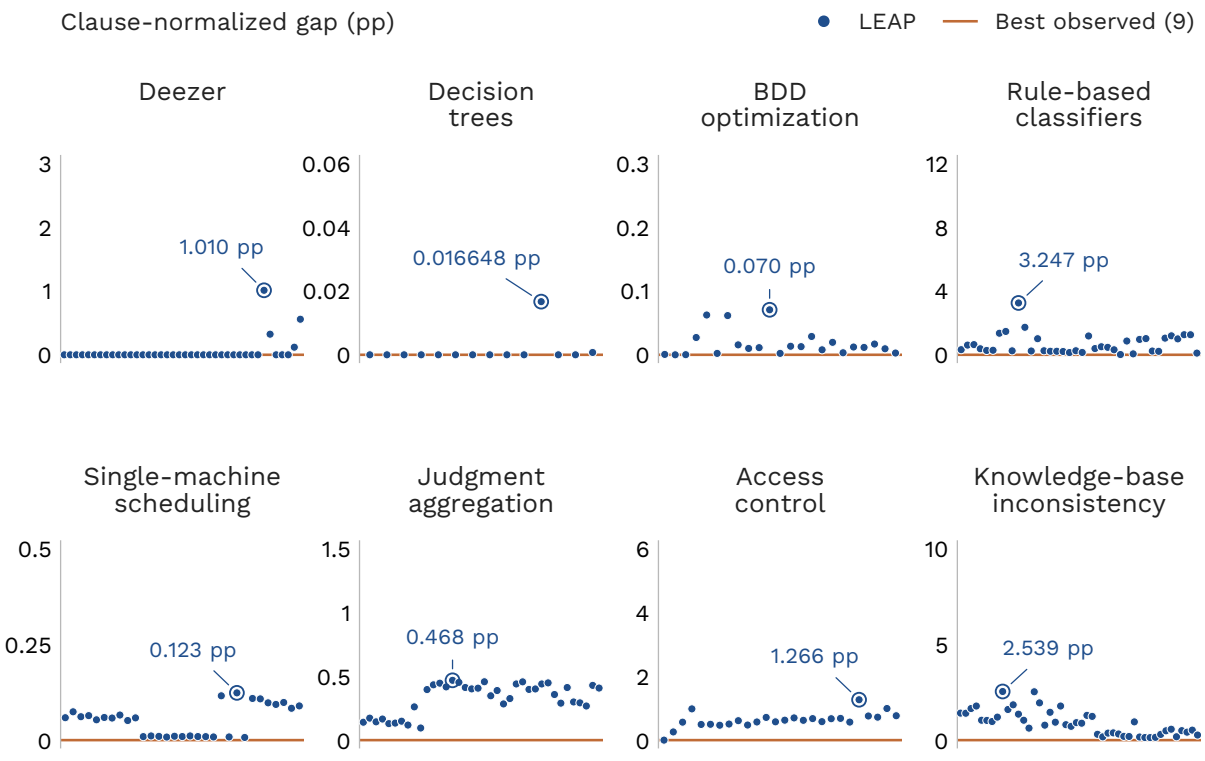


Figure 5: Clause-normalized gaps (percentage points) to the best of nine state-of-the-art solver configurations on each instance. Instances are ordered by size; annotations mark maxima.

7 Concluding Remarks

This paper studies how learned predictions can guide smooth integer programs with provable guarantees. The guarantees operate at two levels: a prediction enters an approximation pipeline with controlled loss, and the oracle is selected by the downstream performance of the algorithm it induces.

Leap realizes the first level by turning a predicted assignment into a linear surrogate, which is optimized by linear programming and rounded. The error-footprint analysis shows that the effect of a prediction depends on both the number of its errors and the part of the objective those errors can affect. The learnability analysis realizes the second level by evaluating each predictor through the Leap algorithm it induces. Experiments support both levels and compare Leap with state-of-the-art solvers on large MAX-SAT application benchmarks.

The computational side of learning remains open. Our analysis gives sample-complexity guarantees for empirical risk minimization over induced Leap algorithms, but does not address whether this ERM problem can be solved efficiently for expressive predictor classes. Tractable procedures for this problem, or surrogate objectives that preserve downstream optimization guarantees, are an important direction for future work.

References

Anders Aamand, Justin Y. Chen, Siddharth Gollapudi, Sandeep Silwal, and Hao Wu. Improved approximations for hard graph problems using predictions. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pages 73–101, Vancouver, Canada, 2025. PMLR.

Martin Anthony and Peter L Bartlett. *Neural network learning: Theoretical foundations*. Cambridge University Press, Cambridge, UK, 1999.

Antonios Antoniadis, Joan Boyar, Marek Elias, Lene Monrad Favrholdt, Ruben Hoeksma, Kim S. Larsen, Adam Polak, and Bertrand Simon. Paging with succinct predictions. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pages 952–968, Honolulu, Hawaii, USA, 2023. PMLR.

Sanjeev Arora, Carsten Lund, Rajeev Motwani, Madhu Sudan, and Mario Szegedy. Proof verification and the hardness of approximation problems. *Journal of the ACM*, 45(3):501–555, 1998.

Sanjeev Arora, David Karger, and Marek Karpinski. Polynomial time approximation schemes for dense instances of NP-hard problems. *Journal of Computer and System Sciences*, 58(1):193–210, 1999.

Idan Attias, Xing Gao, and Lev Reyzin. Learning-augmented algorithms for Boolean satisfiability. arXiv:2505.06146, 2025.

Xingjian Bai and Christian Coester. Sorting with predictions. In *Advances in Neural Information Processing Systems*, volume 36, New Orleans, Louisiana, USA, 2023. Curran Associates, Inc.

Maria-Florina Balcan, Vaishnavh Nagarajan, Ellen Vitercik, and Colin White. Learning-theoretic foundations of algorithm configuration for combinatorial partitioning problems. In *Proceedings of the 30th Conference on Learning Theory*, volume 65, pages 213–274, Amsterdam, Netherlands, 2017. PMLR.

Maria-Florina Balcan, Travis Dick, Tuomas Sandholm, and Ellen Vitercik. Learning to branch. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80, pages 344–353, Stockholm, Sweden, 2018. PMLR.

Maria-Florina Balcan, Siddharth Prasad, Tuomas Sandholm, and Ellen Vitercik. Sample complexity of tree search configuration: Cutting planes and beyond. In *Advances in Neural Information Processing Systems*, volume 34, Virtual, 2021. Curran Associates, Inc.

Maria-Florina Balcan, Dan Deblasio, Travis Dick, Carl Kingsford, Tuomas Sandholm, and Ellen Vitercik. How much data is sufficient to learn high-performing algorithms? *Journal of the ACM*, 71(5):32:1–32:58, 2024.

Adrian Balint and Uwe Schöning. Choosing probability distributions for stochastic local search and the role of make versus break. In *Theory and Applications of Satisfiability Testing – SAT 2012*, volume 7317, pages 16–29, Trento, Italy, 2012. Springer.

Evripidis Bampis, Bruno Escoffier, and Michalis Xefteris. Parsimonious learning-augmented approximations for dense instances of NP-hard problems. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pages 2700–2714, Vienna, Austria, 2024. PMLR.

Peter Bartlett, Piotr Indyk, and Tal Wagner. Generalization bounds for data-driven numerical linear algebra. In *Proceedings of the 35th Conference on Learning Theory*, volume 178, pages 2013–2040, London, UK, 2022. PMLR.

Peter L. Bartlett and Shahar Mendelson. Rademacher and Gaussian complexities: Risk bounds and structural results. *Journal of Machine Learning Research*, 3:463–482, 2002.

Peter L Bartlett, Nick Harvey, Christopher Liaw, and Abbas Mehrabian. Nearly-tight VC-dimension and pseudodimension bounds for piecewise linear neural networks. *Journal of Machine Learning Research*, 20(63):1–17, 2019.

Peng Chen, Hailiang Zhao, Jiaji Zhang, Xueyan Tang, Yixuan Wang, and Shuiguang Deng. Robustifying Learning-Augmented caching efficiently without compromising 1-Consistency. In *Advances in Neural Information Processing Systems*, volume 38, pages 122677–122720, San Diego, California, USA, 2025. Curran Associates, Inc.

Vincent Cohen-Addad, Tommaso d'Orsi, Anupam Gupta, Euiwoong Lee, and Debmalya Panigrahi. Learning-augmented approximation algorithms for maximum cut and related problems. In *Advances in Neural Information Processing Systems*, volume 37, pages 25961–25980, Vancouver, Canada, 2024. Curran Associates, Inc.

Michael Dinitz, Sungjin Im, Thomas Lavastida, Benjamin Moseley, and Sergei Vassilvitskii. Faster matchings via learned duals. In *Advances in Neural Information Processing Systems*, volume 34, Virtual, 2021. Curran Associates, Inc.

Jon C. Ergun, Zhili Feng, Sandeep Silwal, David Woodruff, and Samson Zhou. Learning-augmented k-means clustering. In *Proceedings of the 10th International Conference on Learning Representations*, Virtual, 2022.

Dimitris Fotakis, Michael Lampis, and Vangelis Th. Paschos. Sub-exponential approximation schemes for CSPs: From dense to almost sparse. In *33rd Symposium on Theoretical Aspects of Computer Science (STACS 2016)*, volume 47, pages 37:1–37:14, Orléans, France, 2016. Schloss Dagstuhl–Leibniz-Zentrum für Informatik.

Maxime Gasse, Didier Chételat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 32, Vancouver, Canada, 2019. Curran Associates, Inc.

Suprovat Ghoshal, Konstantin Markarychev, and Yury Markarychev. Constraint satisfaction problems with advice. In *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1202–1221, New Orleans, Louisiana, USA, 2025. SIAM.

Michel X. Goemans and David P. Williamson. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *Journal of the ACM*, 42(6):1115–1145, 1995.

Rishi Gupta and Tim Roughgarden. A PAC approach to application-specific algorithm selection. *SIAM Journal on Computing*, 46(3):992–1017, 2017.

Sungjin Im, Ravi Kumar, Aditya Petety, and Manish Purohit. Parsimonious learning-augmented caching. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162, pages 9588–9601, Baltimore, Maryland, USA, 2022. PMLR.

Christoph Jabs, Dieter Vandesande, Jeremias Berg, Johannes Fichte, George Katsirelos, Ruben Martins, André Schidler, and Dominik Schreiber. MaxSAT evaluation: Complete benchmark collection. Zenodo, 2026. URL `https://zenodo.org/records/21283148`.

Mikhail Khodak, Maria-Florina Balcan, Ameet Talwalkar, and Sergei Vassilvitskii. Learning predictions for algorithms with predictions. In *Advances in Neural Information Processing Systems*, volume 35, New Orleans, Louisiana, USA, 2022. Curran Associates, Inc.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *Proceedings of the 5th International Conference on Learning Representations*, Toulon, France, 2017.

Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. The case for learned index structures. In *Proceedings of the 2018 International Conference on Management of Data*, pages 489–504, Houston, TX, USA, 2018. ACM.

Ravi Kumar, Manish Purohit, and Zoya Svitkina. Improving online algorithms via ML predictions. In *Advances in Neural Information Processing Systems*, pages 9684–9693, Montréal, Canada, 2018. Curran Associates, Inc.

Silvio Lattanzi, Ola Svensson, and Sergei Vassilvitskii. Speeding up Bellman Ford via minimum violation permutations. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pages 18584–18598, Honolulu, Hawaii, USA, 2023. PMLR.

Yujia Li, Daniel Tarlow, Marc Brockschmidt, and Richard Zemel. Gated graph sequence neural networks. In *Proceedings of the 4th International Conference on Learning Representations*, San Juan, Puerto Rico, 2016.

Thodoris Lykouris and Sergei Vassilvitskii. Competitive caching with machine learned advice. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80, pages 3296–3305, Stockholm, Sweden, 2018. PMLR.

Thodoris Lykouris and Sergei Vassilvitskii. Competitive caching with machine learned advice. *Journal of the ACM*, 68(4):24:1–24:25, 2021.

MaxSAT Evaluation. MaxSAT evaluation 2024, 2024. URL `https://maxsat-evaluations.github.io/2024/mse24-talk.pdf`.

Shahar Mendelson and Roman Vershynin. Entropy and the combinatorial dimension. *Inventiones Mathematicae*, 152(1):37–55, 2003.

Michael Mitzenmacher. Scheduling with predictions and the price of misprediction. In *Proceedings of the 11th Innovations in Theoretical Computer Science Conference*, volume 151, pages 14:1–14:18, Seattle, Washington, USA, 2020. Schloss Dagstuhl–Leibniz-Zentrum für Informatik.

Michael Mitzenmacher and Sergei Vassilvitskii. Algorithms with predictions. *Communications of the ACM*, 65(7):33–35, 2022.

Leyan Pan, Vijay Ganesh, Jacob Abernethy, Chris Esposo, and Wenke Lee. Can transformers reason logically? A study in SAT solving. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pages 47632–47671, Vancouver, Canada, 2025. PMLR.

Christos H. Papadimitriou. *Computational Complexity*. Addison-Wesley, Reading, MA, USA, 1994.

Christos H. Papadimitriou and Mihalis Yannakakis. Optimization, approximation, and complexity classes. *Journal of Computer and System Sciences*, 43(3):425–440, 1991.

David Pollard. *Empirical processes: Theory and applications*. Institute of Mathematical Statistics, Hayward, CA, USA, 1990.

Prasad Raghavendra and Ning Tan. Approximating CSPs with global cardinality constraints using SDP hierarchies. In *Proceedings of the 23rd Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 373–387, Kyoto, Japan, 2012. SIAM.

Benedek Rozemberczki, Oliver Kiss, and Rik Sarkar. Karate Club: An API oriented open-source Python framework for unsupervised learning on graphs. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management*, pages 3125–3132, Virtual Event, Ireland, 2020. ACM.

Norbert Sauer. On the density of families of sets. *Journal of Combinatorial Theory, Series A*, 13(1): 145–147, 1972.

Bart Selman, Hector J. Levesque, and David G. Mitchell. A new method for solving hard satisfiability problems. In *Proceedings of the Tenth National Conference on Artificial Intelligence*, pages 440–446, San Jose, California, USA, 1992. AAAI Press.

Bart Selman, Henry A. Kautz, and Bram Cohen. Local search strategies for satisfiability testing. In *Cliques, Coloring, and Satisfiability: Second DIMACS Implementation Challenge*, pages 521–532. American Mathematical Society, Providence, RI, USA, 1996.

Daniel Selsam, Matthew Lamm, Benedikt Bünz, Percy Liang, Leonardo de Moura, and David L. Dill. Learning a SAT solver from single-bit supervision. In *Proceedings of the 7th International Conference on Learning Representations*, New Orleans, Louisiana, USA, 2019.

Rana Shahout and Michael Mitzenmacher. SkipPredict: When to invest in predictions for scheduling. In *Advances in Neural Information Processing Systems*, Vancouver, Canada, 2024. Curran Associates, Inc.

Appendix

This appendix is organized as follows:

- Section A gives extended related-work discussion.
- Section B gives additional experimental details.
- Section C gives omitted approximation-framework proofs, including the randomized rounding analysis.
- Section D gives omitted proofs for the learnability guarantees.
- Section E verifies the smoothness condition for MAX-k-CSP.

A Extended Related Work

A.1 Detailed Comparison with Approximation Results

We compare approximation results through their structural assumptions, advice models, and guarantees. Throughout this subsection, $q = \Pr[\text{advice bit correct}] - 1/2$, $\varepsilon = \|\widehat{\mathbf{x}} - \mathbf{x}^*\|_1$, and r denotes the fraction of variables revealed by subset advice.

Classical Approximation and Density For general MAX-CUT without advice, Goemans and Williamson (1995) achieve approximation ratio $\alpha_{\mathrm{GW}} \approx 0.87856$ by solving an SDP relaxation and applying random-hyperplane rounding. Density assumptions permit stronger guarantees. For dense MAX-CUT and MAX-k-SAT, where the optimum is $\Omega(n^2)$ and $\Omega(n^k)$, respectively, Arora et al. (1999) obtain a $(1-\eta)$-approximation for any fixed $\eta > 0$ through exhaustive sampling, LP relaxation, and rounding. The running time is commonly written as $n^{O(1/\eta^2)}$, with density parameters suppressed in the exponent. Fotakis et al. (2016) extend this approach to almost-sparse MAX-k-CSP instances with $\mathrm{OPT} = \Omega(n^{k-1+\gamma})$ for fixed $\gamma \in (0, 1]$, obtaining a $(1-\eta)$-approximation in $2^{O(n^{1-\gamma}\log n/\eta^3)}$ time.

Sampled Advice Bampis et al. (2024) incorporate predictions into the sampling framework through a parsimonious one-bit oracle, queried on a sampled set S. For ρ-dense MAX-CUT, the prediction-dependent ratio is $1 - \eta - 8\,\mathrm{err}/(\rho|S|)$, where err counts incorrect predictions in S. Their running time is $O(nT_{\mathrm{LP}})$, where T_{LP} is the time to solve an LP with n variables and $O(n)$ constraints. For general smooth integer programs of degree d and smoothness parameter β, their method requires $O(n\log(\beta n d))$ LP solves.

Label and Edge Advice Cohen-Addad et al. (2024) obtain an $\alpha_{\mathrm{GW}} + \widetilde{\Omega}(q^4)$ approximation for MAX-CUT from label advice with bias q, with pairwise independence sufficient for their guarantee. Under independent label advice and sufficiently large average degree $\bar{d} = 2|E|/n$, Ghoshal et al. (2025) obtain expected value at least $(1 - O(1/(q\sqrt{\bar{d}})))\mathrm{OPT}$ for unweighted MAX-CUT and Max-2-Lin. With independent q-correlated edge advice, Aamand et al. (2025) achieve $\alpha_{\mathrm{GW}} + \widetilde{\Omega}(q^2)$ for MAX-CUT by aggregating predictions at high-degree vertices and using an approximation subroutine for the remaining part.

Subset Advice Subset advice reveals the optimal labels of a random fraction r of variables. For MAX-CUT, Cohen-Addad et al. (2024) obtain expected ratio $\alpha_{\mathrm{RT}} + \Omega(r)$, where $\alpha_{\mathrm{RT}} \approx 0.858$ is the max-bisection ratio of Raghavendra and Tan (2012). For MAX-SAT, Attias et al. (2025) show that simplifying the formula with these labels and applying an α-approximation to the remaining instance yields ratio $\alpha + (1 - \alpha)r$.

Relationship to Footprint Analysis Our framework uses a full assignment prediction and controls its effect through the footprint budget $\Phi_\varepsilon(\hat{\mathbf{x}})$, without requiring independence of prediction errors. For MAX-CUT ($k = 2$) and MAX-k-SAT, global smoothness gives ratio $1 - O(\varepsilon/n^\gamma)$ when $\mathrm{OPT} = \Omega(n^{k-1+\gamma})$, and the local bound gives $1 - O(\Delta\varepsilon/n)$ when each vertex has degree at most Δ or each literal occurs in at most Δ clauses. With the doubling grid of proposition 3.1, Leap requires $O(\log n)$ LP solves and retains these approximation orders.

A.2 Broader Learning-Augmented Algorithms

Learning-augmented algorithms use predictions to improve performance while studying consistency, degradation with prediction error, and robustness (Lykouris and Vassilvitskii, 2021; Mitzenmacher and Vassilvitskii, 2022). Applications include scheduling (Kumar et al., 2018; Mitzenmacher, 2020; Shahout and Mitzenmacher, 2024), matching (Dinitz et al., 2021), sorting (Bai and Coester, 2023), clustering (Ergun et al., 2022), indexing (Kraska et al., 2018), shortest paths (Lattanzi et al., 2023), paging (Antoniadis et al., 2023), caching (Lykouris and Vassilvitskii, 2018; Im et al., 2022; Chen et al., 2025), and branch-and-bound (Balcan et al., 2018).

B Additional Experimental Details

This appendix provides construction and protocol details for the experiments in section 6.

B.1 Controlled Perturbation Experiments

For each prediction in either perturbation experiment, we enumerate all integer budgets $b \in \{0, \ldots, n\}$. For each budget, we solve the footprint relaxation of (d-LP) and apply greedy rounding in natural variable order, rounding ties to 1. We return the rounded assignment with the largest original objective value, breaking ties in favor of the smaller budget. For both constructions, the plotted flip fractions are $0\%, 1\%, 2\%, 5\%, 10\%, 20\%$.

Dense constructions. For MAX-CUT, we draw a balanced planted bipartition $\mathbf{x}^*$ and sample $\lceil 0.1n^2 \rceil$ crossing edges, so that $\mathrm{OPT} = |E|$. For MAX-SAT, we draw a planted assignment $\mathbf{x}^*$ and sample $\lceil 0.002n^3 \rceil$ distinct 3-clauses satisfied by $\mathbf{x}^*$, so that $\mathrm{OPT} = |\mathcal{C}|$. We use five independent seeds per problem and size. Within each instance, we fix one random variable ordering and flip progressively longer prefixes of this ordering to create nested perturbations.

Sparse constructions. For MAX-CUT, we sample $2n$ edges across a planted random bipartition, using nonuniform vertex sampling to obtain heterogeneous degrees. For MAX-SAT, we generate $2n$ distinct planted-satisfied 3-clauses, with each variable occurring at most eight times. Thus both constructions have $\mathrm{OPT} = 2n$. For each problem and size in fig. 3, we generate five independent planted instances. Within each instance, we average four perturbations at each flip fraction: two random flip sets, a highest-load set, and a lowest-load set. Load is graph degree for MAX-CUT and variable occurrence count for MAX-SAT. Curves average the five instance-level means.

B.2 Empirical Selection over Oracle Candidates

For each train-test setting, we use 3000 training instances and 2000 test instances. MAX-CUT instances are Barabási–Albert graphs, and MAX-SAT instances are random 3-CNF formulas with a fixed clause-to-variable ratio. Cost counts uncut edges for MAX-CUT and unsatisfied clauses for MAX-SAT in the assignment returned by Leap. For MAX-CUT, CNF-based candidates use an XOR-CNF encoding, in which each edge (u, v) is represented by clauses enforcing $x_u \neq x_v$; all assignments are scored under the original graph objective.

Table 3 summarizes the four candidate classes. Each local-search variant uses time budgets of $0.0025, 0.005, 0.01, 0.025$ seconds. For each neural architecture, checkpoints after $1, 2, 4$ training epochs define the candidates. The neural training instances are random 3-CNF formulas for MAX-SAT and XOR-CNF encodings of Barabási–Albert graphs for MAX-CUT.

Table 3: Oracle candidates within each class.

Class	Variants	Count
Annealing SLS	Simulated annealing (Kirkpatrick et al., 1983)	4
WalkSAT-style SLS	GSAT, MaxWalkSAT, weighted and restart MaxWalkSAT (Selman et al., 1992, 1996)	16
ProbSAT-style SLS	ProbSAT-like and SATLike-style search (Balint and Schöning, 2012)	8
GNN predictors	GGNN, NeuroSAT, and GCN (Li et al., 2016; Selsam et al., 2019; Kipf and Welling, 2017)	9

For each problem, oracle class, and training size m, we repeat the training draw 50 times and report means and standard errors.

Selection at a Fixed Instance Size. Figures 6 to 8 show results for $n = 100, 500, 1000$. These plots generally show the selected candidates' excess risk declining toward zero as sample counts increase, though some families exhibit intermediate fluctuations.

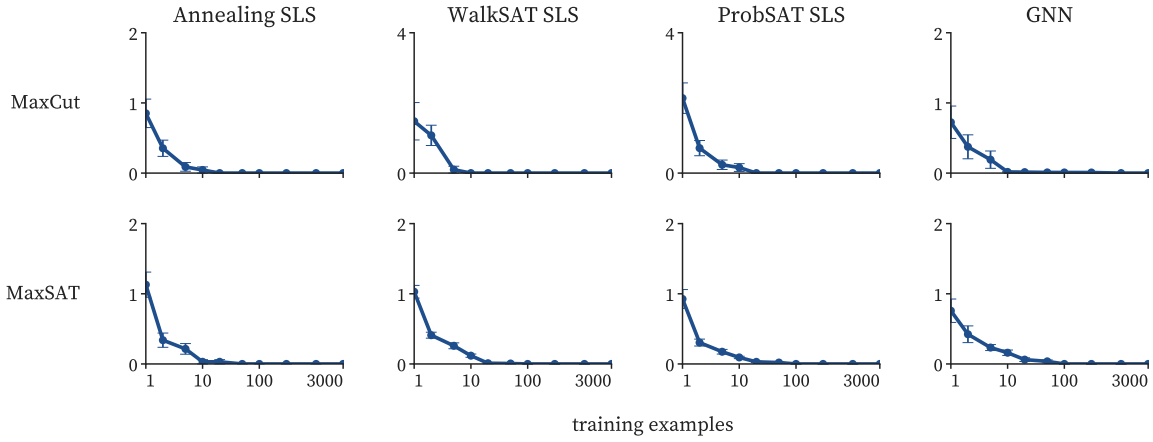


Figure 6: Additional empirical-selection results for $n = 100$. The horizontal axis shows the number of training instances; the vertical axis shows excess test risk.

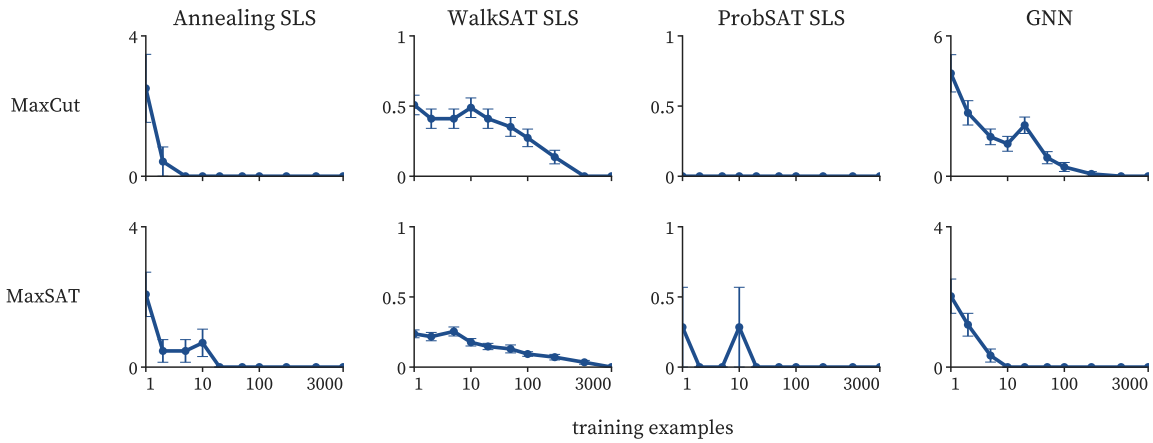


Figure 7: Additional empirical-selection results for $n = 500$. The horizontal axis shows the number of training instances; the vertical axis shows excess test risk.

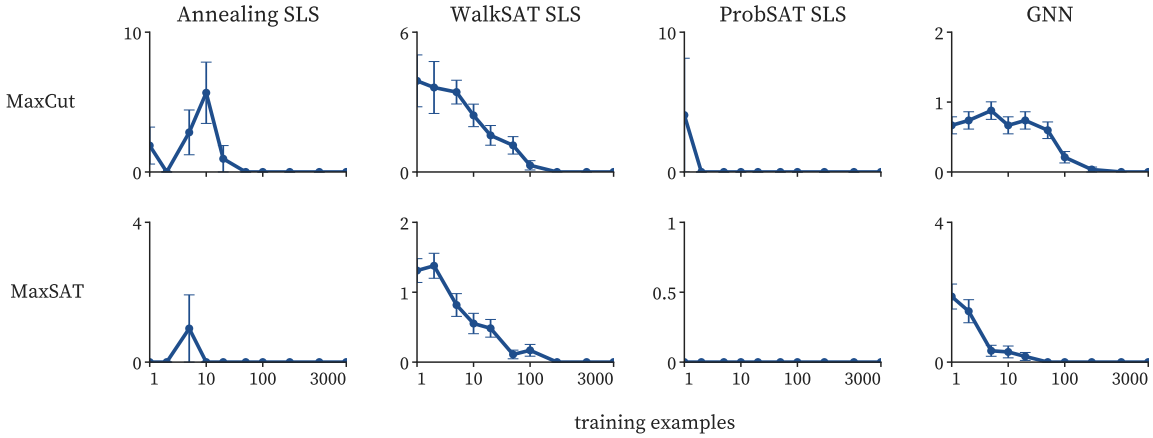


Figure 8: Additional empirical-selection results for $n = 1000$. The horizontal axis shows the number of training instances; the vertical axis shows excess test risk.

Selection across Instance Sizes. We evaluate candidates selected on smaller training instances and tested on larger instances. Figures 9 and 10 show results for extrapolation. The selected candidates approach zero excess risk, though a persistent positive gap is observed for the ProbSAT family when transferring from $n = 100$ to $n = 200$ in MAX-SAT.

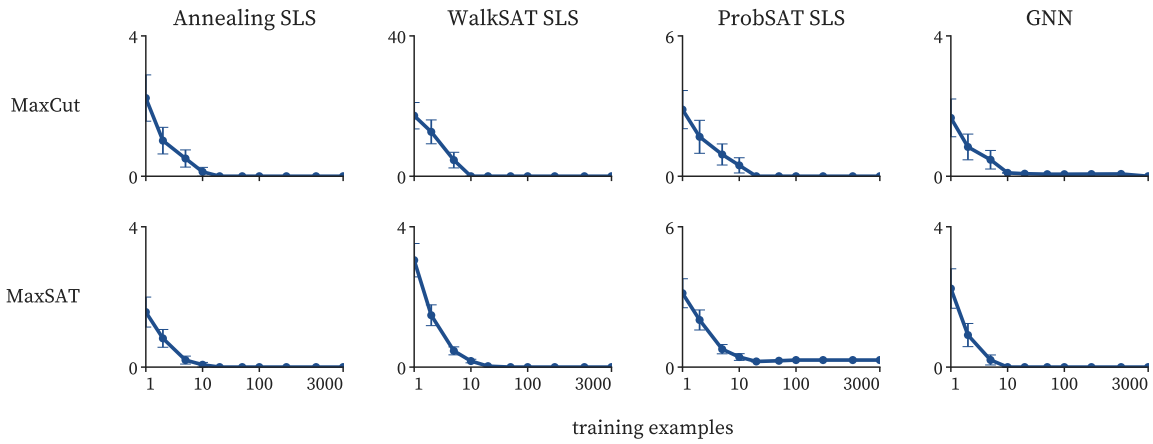


Figure 9: Empirical-selection results for extrapolation from $n = 100$ to $n = 200$. The horizontal axis shows the number of training instances; the vertical axis shows excess test risk on the larger test instances.

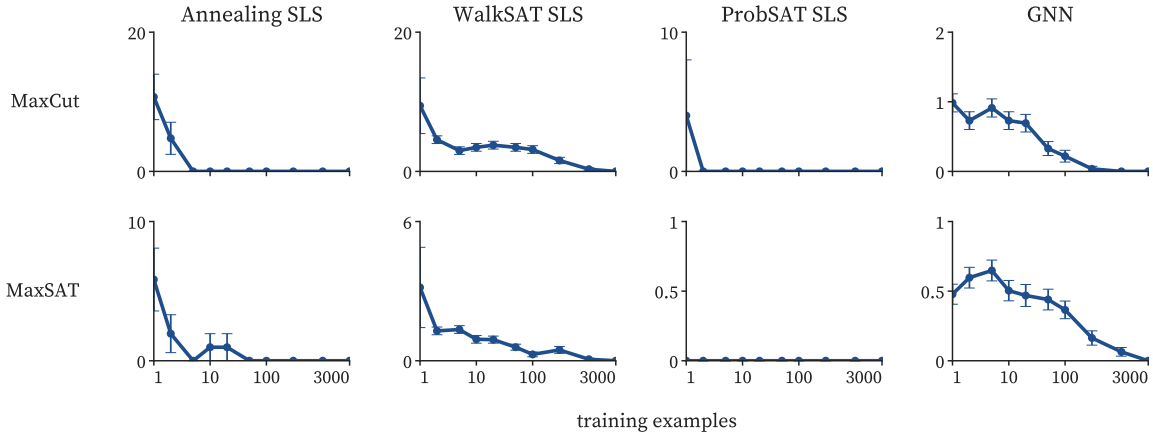


Figure 10: Empirical-selection results for extrapolation from $n = 500$ to $n = 1000$. The horizontal axis shows the number of training instances; the vertical axis shows excess test risk on the larger test instances.

B.3 Details of the Large-Scale MAX-SAT Benchmarks

Seven families are drawn from the historical anytime-unweighted or incomplete-unweighted tracks of the MAX-SAT Evaluation archive (Jabs et al., 2026); the eighth consists of 40 Deezer ego networks from SNAP (Rozemberczki et al., 2020). Table 4 describes their source applications. After deduplication by normalized formula content, we retain 261 instances. The common comparison contains 256 instances; five BDD instances lack assignments from at least one baseline.

All methods use the same formula. For Deezer, the XOR-CNF encoding of each edge makes the cost the number of uncut edges and gives $M = 2|E|$. Family means average the per-instance clause-normalized gaps.

We use the MSE 2024 versions of Exact, Loandra, NuWLS-c-IBR, SPB-MaxSAT-c, SPB-MaxSAT-c-Band, SPB-MaxSAT-c-FPS, noSAT-MaxSATv3, and the Glucose and IntelSAT variants of tt-open-wbo-inc (MaxSAT Evaluation, 2024). Baselines receive 60 seconds and 12 GiB per instance. LEAP performs ten rounds from each of five random starts, using the output of each round

Table 4: Source applications of the eight benchmark families. Their encodings are evaluated as unit-weight all-soft formulas.

Family	**Source application**
Deezer	Partition social ego networks to maximize crossing edges.
Decision trees	Learn accurate decision trees under depth and size bounds.
BDD optimization	Learn bounded-depth binary decision diagram classifiers.
Rule-based classifiers	Learn Boolean rules balancing classification accuracy and complexity.
Single-machine scheduling	Maximize on-time jobs subject to release times, processing requirements, and dependencies.
Judgment aggregation	Find consistent collective rankings minimizing disagreement with voter or majority preferences.
Access control	Select roles granting required permissions under permission and incompatibility limits.
Knowledge-base inconsistency	Quantify inconsistency through violated statements, contradictory values, or distances to satisfying assignments.

as the prediction for the next, and returns the best final assignment by cost. Each round uses the doubling budget grid of proposition 3.1.

C Omitted Material for the Approximation Framework

This section presents technical details supporting the approximation framework stated in section 3.

We first record the concentration bound used for randomized rounding of linear objectives.

Lemma C.1. *Let* $\mathbf{y} \in [0,1]^n$*, and let* $\mathbf{z} \in \{0,1\}^n$ *be obtained by independent randomized rounding, where* $\Pr[z_i = 1] = y_i$ *for all* $i \in [n]$*. Suppose the coefficients* $(c_i)_{i=1}^n$ *satisfy* $|c_i| \le \beta$*. Then, for any* $s \ge 1$*, with probability at least* $1 - 2/n^{s+1}$*,*

$$\Big|\sum_{i=1}^{n} c_i(y_i - z_i)\Big| \le \beta\sqrt{\tfrac{s+1}{2}}\,\sqrt{n\ln n}.$$

Proof. Let $X = \sum_{i=1}^n c_i(y_i - z_i)$. Since $\mathbb{E}[z_i] = y_i$, we have $\mathbb{E}[X] = 0$, and changing one z_i changes X by at most $|c_i| \le \beta$. McDiarmid's inequality gives, for any $t > 0$,

$$\Pr\left(|X| \ge t\right) \le 2\exp\left(-\frac{2t^2}{n\beta^2}\right).$$

Choosing $t = \beta\sqrt{\frac{s+1}{2}}\,\sqrt{n\ln n}$ yields the claim with probability at least $1 - 2/n^{s+1}$. □

Lemma 3.4 (Randomized rounding for smooth objectives)**.** *Let* $\mathbf{y} \in [0,1]^n$*, and let* $\mathbf{z} \in \{0,1\}^n$ *be generated via independent randomized rounding where* $\Pr[z_i = 1] = y_i$ *for all* $i \in [n]$*. Consider an* n*-variate degree-*d *polynomial* $p(\mathbf{x})$ *that is* β*-smooth. For any* $s > d$*, with probability at least* $1 - 2d/n^{s-d+2}$*,*

$$|p(\mathbf{y}) - p(\mathbf{z})| \le \eta_d \beta n^{d-1} \sqrt{\frac{s+1}{2}} \sqrt{n \ln n},$$

where η_d *is a constant depending only on* d*.*

Proof. Let

$$t = \sqrt{\tfrac{s+1}{2}} \sqrt{n \ln n}.$$

We prove the statement by induction on the degree ℓ of a smooth component. For every degree-ℓ component r, there is a constant η_ℓ depending only on ℓ such that

$$|r(\mathbf{z}) - r(\mathbf{y})| \le \eta_\ell \beta n^{\ell-1} t$$

with failure probability at most $2\ell/n^{s-\ell+2}$. The base case $\ell = 1$ is exactly lemma C.1. For the induction step, write $r(\mathbf{x}) = c + \sum_i x_i r_i(\mathbf{x})$, where each r_i has degree at most $\ell - 1$. Then

$$|r(\mathbf{z}) - r(\mathbf{y})| \le \Big| \sum_i z_i (r_i(\mathbf{z}) - r_i(\mathbf{y})) \Big| + \Big| \sum_i (z_i - y_i) r_i(\mathbf{y}) \Big|.$$

For the first term, the induction hypothesis applied to all n child components and a union bound give a contribution at most $n\eta_{\ell-1}\beta n^{\ell-2} t$, with failure probability at most $2(\ell-1)/n^{s-\ell+2}$. For the second term, lemma 3.2 bounds $|r_i(\mathbf{y})| \le 2e\beta n^{\ell-1}$, and lemma C.1 gives

$$\Big| \sum_i (z_i - y_i) r_i(\mathbf{y}) \Big| \le 2e\beta n^{\ell-1} t$$

with failure probability at most $2/n^{s+1}$. Absorbing constants into η_ℓ and union-bounding the two failure events proves the induction step. Taking $\ell = d$ proves the lemma. □

D Omitted Material for Learnability Guarantees

We recall pseudo-dimension and derive from uniform convergence the ERM guarantee used in section 4.

Definition D.1 (Pseudo-dimension, Anthony and Bartlett 1999)**.** Let $\mathcal{H}$ be a set of real-valued functions defined on Π. A finite subset $S = \{\pi_1, \dots, \pi_m\} \subseteq \Pi$ is *(pseudo-)shattered* by $\mathcal{H}$ if there exist real-valued witnesses $r_1, \dots, r_m$ such that for each subset $T \subseteq S$, there exists a function $h \in \mathcal{H}$ satisfying $h(\pi_i) > r_i$ if and only if $\pi_i \in T$. The *pseudo-dimension* of $\mathcal{H}$, denoted by $\mathrm{Pdim}(\mathcal{H})$, is the maximum cardinality m for which some subset $S \subseteq \Pi$ of size m is pseudo-shattered by $\mathcal{H}$.

Lemma D.1 (Uniform convergence)**.** *Let* $H > 0$*, and let* $\mathcal{H}$ *be a nonempty class of functions mapping* Π *to* $[0, H]$*, with finite pseudo-dimension* $d_{\mathcal{H}}$*. Assume the suprema below are measurable. For any distribution* $\mathcal{D}$ *over* Π*, any precision* $\alpha > 0$*, and any confidence parameter* $\delta \in (0, 1]$*, if the sample size satisfies*

$$m \ge C \left(\frac{H}{\alpha}\right)^2 \left(d_{\mathcal{H}} + \log\left(\frac{2}{\delta}\right)\right), \tag{1}$$

where C *is an absolute constant, then with probability at least* $1 - \delta$ *over an i.i.d. sample* $(\pi_1, \dots, \pi_m) \sim \mathcal{D}^m$*, the following uniform bound holds:*

$$\sup_{h \in \mathcal{H}} \left| \mathbb{E}_{\pi \sim \mathcal{D}}[h(\pi)] - \frac{1}{m} \sum_{i=1}^{m} h(\pi_i) \right| \le \alpha.$$

Proof. Let $\mathcal{G} = \{h/H : h \in \mathcal{H}\}$. Its fat-shattering dimension at every positive scale is at most $d_{\mathcal{H}}$. By Mendelson and Vershynin (2003, Theorem 1), its $L_2(Q)$ covering number satisfies

$$\log N(t, \mathcal{G}, L_2(Q)) \le C_0 d_{\mathcal{H}} \log(2/t) \qquad (0 < t < 1)$$

for every probability measure Q. For a sample $S = (\pi_1, \dots, \pi_m)$, let Q_S be its empirical measure and let $\sigma_1, \dots, \sigma_m$ be independent uniform signs in $\{-1, 1\}$. Dudley's entropy integral bounds the empirical Rademacher complexity as

$$\begin{aligned}\widehat{\mathfrak{R}}_S(\mathcal{G}) &\triangleq \mathbb{E}_\sigma\left[\sup_{g \in \mathcal{G}} \frac{1}{m} \sum_{i=1}^{m} \sigma_i g(\pi_i)\right] \\ &\le \frac{C_1}{\sqrt{m}} \int_0^1 \sqrt{\log N(t, \mathcal{G}, L_2(Q_S))}\, dt \le C_2 \sqrt{\frac{d_{\mathcal{H}}}{m}},\end{aligned}$$

since $\int_0^1 \sqrt{\log(2/t)}\, dt$ is finite. Symmetrization, bounded differences, and a union bound over the two one-sided deviations give a uniform deviation of at most $C_3 H \sqrt{(d_{\mathcal{H}} + \log(2/\delta))/m}$ with probability at least $1 - \delta$ (Bartlett and Mendelson, 2002). All constants are absolute. The sample size in (1) makes this bound at most α. □

The following lemma records the ERM consequence used in the main text.

Lemma D.2. *Fix $H > 0$, $\alpha > 0$, $\delta \in (0, 1]$, instance space Π, and cost function $\texttt{COST} : \mathcal{A} \times \Pi \to [0, H]$. Suppose the algorithm class $\mathcal{A}$ induces a cost function class with pseudo-dimension at most $d_{\mathcal{A}}$. If m satisfies (1) with $d_{\mathcal{H}} = d_{\mathcal{A}}$, then with probability at least $1 - \delta$, any empirical risk minimizer $\hat{A}$ satisfies*

$$\mathbb{E}_{\pi \sim \mathcal{D}}[\texttt{COST}(\hat{A}, \pi)] - \min_{A \in \mathcal{A}} \mathbb{E}_{\pi \sim \mathcal{D}}[\texttt{COST}(A, \pi)] \le 2\alpha.$$

Proof. Let $\mathcal{H} = \{h_A : \Pi \to [0, H] \mid A \in \mathcal{A}\}$, where $h_A(\pi) := \texttt{COST}(A, \pi)$. By assumption, $\mathrm{Pdim}(\mathcal{H}) \le d_{\mathcal{A}}$. If m satisfies (1), then lemma D.1 gives, with probability at least $1 - \delta$,

$$\sup_{A \in \mathcal{A}} \left| \mathbb{E}_{\pi \sim \mathcal{D}}[h_A(\pi)] - \tfrac{1}{m} \sum_{i=1}^{m} h_A(\pi_i) \right| \le \alpha.$$

Let $A_{\mathcal{D}} \in \arg\min_{A \in \mathcal{A}} \mathbb{E}_{\pi \sim \mathcal{D}}[h_A(\pi)]$ be the optimal algorithm, and let $\hat{A}$ be the ERM solution minimizing $\frac{1}{m} \sum_{i=1}^{m} h_A(\pi_i)$. Then

$$\begin{aligned}\mathbb{E}_{\pi \sim \mathcal{D}}[h_{\hat{A}}(\pi)] &\le \tfrac{1}{m} \sum_{i=1}^{m} h_{\hat{A}}(\pi_i) + \alpha \\ &\le \tfrac{1}{m} \sum_{i=1}^{m} h_{A_{\mathcal{D}}}(\pi_i) + \alpha \\ &\le \mathbb{E}_{\pi \sim \mathcal{D}}[h_{A_{\mathcal{D}}}(\pi)] + 2\alpha,\end{aligned}$$

where the first and third inequalities follow from uniform convergence, and the second from the definition of ERM. □

E Smoothness of MAX-k-CSP

This section verifies the smoothness condition for the standard polynomial formulation of MAX-k-CSP.

Every problem in MAX-SNP L-reduces to MAX-k-CSP for some constant k (Papadimitriou and Yannakakis, 1991; Papadimitriou, 1994). Specific graph problems such as MAX-CUT admit low-degree polynomial formulations, as in example 1.

We represent MAX-k-CSP as a smooth integer program of degree k by arithmetization. Consider an instance with n decision variables $x_1, \ldots, x_n \in \{0,1\}$ and m constraints. Each constraint $j \in [m]$ is defined on a k-variable scope and is specified by a Boolean function $c_j : \{0,1\}^k \to \{0,1\}$.

We encode each constraint as a polynomial. For a constraint c on variables $(x_{i_1}, \ldots, x_{i_k})$, define $t_c(\mathbf{x})$ as the sum of indicator polynomials over its satisfying assignments. For each assignment $a \in \{0,1\}^k$ with $c(a) = 1$, the indicator polynomial is

$$\delta_a(\mathbf{x}) = \prod_{r=1}^{k} x_{i_r}^{a_r}(1 - x_{i_r})^{1-a_r}.$$

Then, $t_c(\mathbf{x}) = \sum_{a:c(a)=1} \delta_a(\mathbf{x})$. The objective is the sum over all constraints:

$$p(\mathbf{x}) = \sum_{j=1}^{m} t_{c_j}(\mathbf{x}).$$

This function is a polynomial of degree at most k.

To bound the smoothness parameter, let M be the maximum number of constraints defined on any single k-variable scope.

- For MAX-CUT on simple graphs, any pair of vertices has at most one edge, so $M = 1$.
- For MAX-k-SAT, multiple clauses may involve the same set of variables, for example $x_1 \vee x_2 \vee x_3$ and $\neg x_1 \vee x_2 \vee x_3$. If duplicate identical clauses are excluded, then $M \leq 2^k$.

The following lemma bounds the smoothness parameter in terms of M and k.

Lemma E.1. *The polynomial objective $p(\mathbf{x})$ derived from a MAX-k-CSP instance is β-smooth for any $\beta \geq M2^k$.*

Proof. We bound each monomial coefficient in the expansion of $p(\mathbf{x})$.

For any single constraint c on a set of k variables, the polynomial $t_c(\mathbf{x})$ is a sum of at most 2^k terms of the form $\delta_a(\mathbf{x})$. The expansion of each $\delta_a(\mathbf{x})$ produces monomials with coefficients ± 1. Hence every monomial coefficient contributed by a single constraint has magnitude at most 2^k.

A monomial on a set S of $\ell \leq k$ variables receives contributions only from constraints whose scope contains S. There are $\binom{n-\ell}{k-\ell} \leq n^{k-\ell}$ such scopes, each carrying at most M constraints, so the coefficient of this monomial has magnitude at most $M2^k n^{k-\ell} \leq \beta n^{k-\ell}$. □